# Sensors & Systems

Ranging-based Sensors

Jan Bendtsen, Torben Knudsen | Electronic Systems, 8th Semester

## Section 1 Ranging-based Sensors & Applications

A **ranging-based sensor** measures the *distance* from the sensor to a target or surface. Rather than capturing an image or measuring a physical quantity at the sensor's own location, it sends out some kind of wave or pulse, waits for the echo to return, and infers distance from the round-trip travel time or phase. The three dominant technologies are **Sonar**, **Radar**, and **LiDAR** — each using a different part of the wave spectrum.


Figure 1: Overview of the three main ranging sensor families: Radar (radio waves, long range, penetrates weather), LiDAR (laser light, high spatial resolution), and Sonar (sound waves, primarily underwater).

### 1.1 Sonar — Sound Navigation and Ranging

**Sonar** uses *sound propagation* to navigate and measure distances. It is most commonly used *underwater* (submarines, ship navigation, fish finding), where radio waves cannot propagate effectively.


Figure 2: Active sonar (left) emits a sound pulse and listens for echoes returning from objects. Passive sonar (right) only listens for sounds emitted by other vessels or targets.

There are two operating modes:

- **Passive sonar** — simply listens for sounds emitted by other objects (e.g. an enemy submarine's engine noise). No pulse is transmitted, so the sensor itself remains undetected.
- **Active sonar** — emits a pulse of sound and listens for the returning echo. Allows precise distance measurement but reveals the presence of the sensor.

**Nature uses sonar too.** Bats, toothed whales (e.g. dolphins), and certain shrews use *active* biological sonar (echo-location) to navigate and hunt. Dolphins can emit up to 600 clicks per second. The focused beam is shaped by a fatty organ called the *melon*.

## 1.2 Radar — Radio Detection and Ranging

**Radar** uses *radio waves* to determine the distance, direction (azimuth and elevation angles), and *radial velocity* of objects relative to the sensor site.

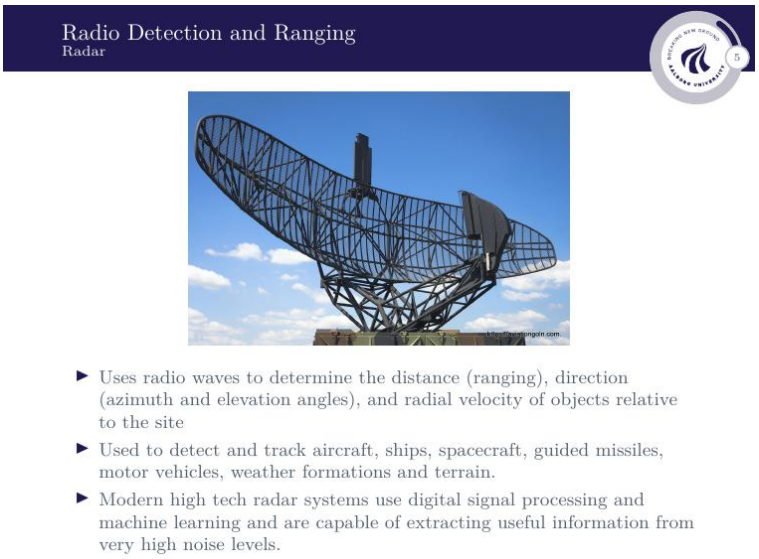


Figure 3: A traditional rotating-dish radar antenna. Modern high-tech radar systems use digital signal processing and machine learning to extract useful information even at very high noise levels.

Key properties of Radar:

- Uses radio waves, typically in the range **30 cm – 3 mm** wavelength (microwave band).
- Penetrates **fog, clouds, rain, and darkness** — making it ideal for aviation, maritime, and automotive applications where LiDAR would struggle in bad weather.
- Can also measure **radial velocity** directly via the Doppler effect (frequency shift of the returning wave).
- Used to detect and track aircraft, ships, spacecraft, guided missiles, motor vehicles, weather formations, and terrain.

**What do azimuth, elevation, and radial velocity actually mean?**

A radar measures three geometric quantities about a target simultaneously:

- **Azimuth** — the *horizontal* angle to the target, like a compass bearing. Standing at the radar: 0° = North, 90° = East, 180° = South, and so on. The rotating dish sweeps through all azimuth angles to scan the full horizon.
- **Elevation** — the *vertical* angle above the horizon. 0° is flat along the ground, 90° is straight up. Together, azimuth + elevation give the full 3D *direction* to the target (like spherical coordinates).
- **Radial velocity** — the component of the target's velocity that is directed *directly toward or away from the radar*. This is measured via the **Doppler effect**: a target moving toward the radar compresses the returning radio wave (higher frequency); a target moving away stretches it (lower frequency). The frequency shift is directly proportional to the radial velocity.

  *Important:* this is *not* the full velocity vector. A car driving sideways past the radar

has nearly zero radial velocity even if it is travelling at 100 km/h — the beam sees it moving neither toward nor away.

Together: **azimuth + elevation + range** give the full 3D *position* of the target; **radial velocity** gives how fast it is approaching or receding. This is why radar is so effective for tracking aircraft: one measurement gives both position and approach speed.

**Active vs. passive radar — and how range is actually computed**

Just like sonar, radar can operate in two modes:

- **Active radar** — transmits its own radio pulse and listens for the returning echo. The classic rotating-dish radar is active.
- **Passive radar** — transmits *nothing*. Instead it exploits existing radio signals in the environment — FM radio stations, TV broadcasts, WiFi, mobile towers — as *illuminators of opportunity*, and detects their reflections off targets. Because no pulse is emitted, the radar itself stays completely undetectable; similar in spirit to passive sonar.

**How range is computed** depends on the radar type:

- **Pulsed radar** — emits a short burst and measures the round-trip *time of flight*. Same principle as pulsed LiDAR:

  $$R = \tfrac{1}{2}\, c\, t$$

  where $c$ is the speed of light and $t$ is the measured delay.
- **FMCW radar** (Frequency-Modulated Continuous Wave) — transmits a continuous signal whose frequency is swept linearly up and down in a ramp (a "chirp"). Because the received echo is delayed relative to the transmitted signal, there is a constant *frequency difference* $\Delta f$ between what is being transmitted and what is being received at any instant. This frequency difference is directly proportional to range:

  $$R \propto \Delta f$$

  FMCW is widely used in automotive radar because it can measure range and radial velocity *simultaneously* from the same waveform.
- **Doppler shift** — the *change* in frequency of the returned wave (not the difference described above) gives the radial velocity, not the range directly.

In summary: **time of flight** → range (pulsed); **frequency difference** → range (FMCW); **frequency shift** → radial velocity (Doppler).

### 1.3 LiDAR — Light Detection and Ranging

**LiDAR** uses *laser light* (ultraviolet, visible, or near-infrared) to measure distances with very high spatial resolution.

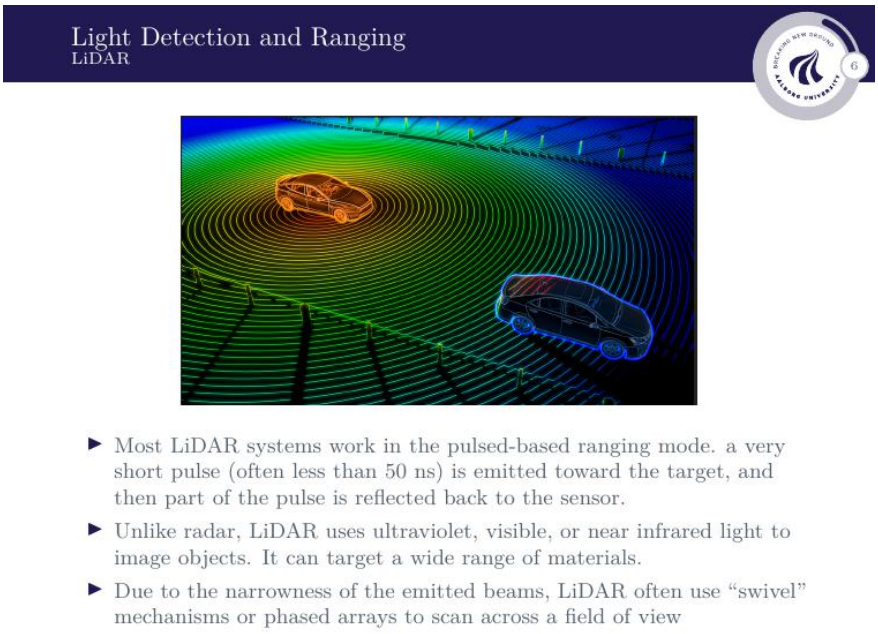


Figure 4: LiDAR point cloud of two cars. A spinning LiDAR head sweeps laser beams in all directions; each reflection gives one 3D data point, building up a dense point cloud of the environment.

Key properties of LiDAR:

- Most systems work in **pulsed** ranging mode: a very short pulse ($< 50\,\text{ns}$) is emitted, and part of the reflected pulse returns to the sensor. Distance is computed from the time of flight.
- Unlike radar, LiDAR uses **laser light** (typically 903 or 905 nm near-infrared), giving *much finer angular resolution* but poorer performance in rain, fog, or dust.
- Due to the narrow beam, LiDAR uses **swivel mechanisms or phased arrays** to scan across a field of view, producing a full 3D point cloud.

**What a returning laser pulse actually tells you — range and colour**

Every time the laser fires, two independent pieces of information come back from the returning pulse:

- **When** it returns $\rightarrow$ **range.** The round-trip time of flight $t$ gives the distance directly:

  $$R = \tfrac{1}{2}\, c\, t$$

  This is exactly the same principle as pulsed radar and pulsed sonar — only the wave speed $c$ differs.

- **How strongly** it returns $\rightarrow$ **reflectivity (intensity / "colour").** The *amplitude* of the returned pulse tells you how much laser energy the surface reflected back toward the sensor. A highly reflective surface (white road markings, metal signs, retroreflectors) returns a strong pulse $\rightarrow$ bright/high-intensity point. A dark or absorbing surface (black asphalt, rubber tyres) returns a weak pulse $\rightarrow$ dim/low-

intensity point. This is why point clouds can be *coloured by return intensity* — each point carries both its 3D position and a reflectivity value. The coloured-intensity image from slide 19 is exactly this: brighter pixels are more reflective surfaces.

**Why wavelength (frequency) matters for material identification.** Different materials reflect different wavelengths differently. LiDAR typically uses near-infrared (903–905 nm) because:

- It is eye-safe at reasonable power levels.
- Water and chlorophyll *absorb* near-infrared, so vegetation gives a weaker return than dry surfaces at the same distance — the sensor can start to distinguish material type from intensity alone.
- Using *two wavelengths simultaneously* (multi-spectral LiDAR) gives two intensity values per point. Because each material has its own reflectivity "fingerprint" across wavelengths, comparing the two intensities allows the system to *classify* what the surface is made of (e.g. tree canopy vs. ground vs. building facade) even if they are at the same height.

In short: **time** → range; **amplitude** → reflectivity / point colour; **wavelength choice** → material sensitivity.

**Sensor comparison at a glance**

| Sensor | Wave type | Range | Resolution | Weather |
|---|---|---|---|---|
| Sonar | Sound (∼kHz–MHz) | Medium (water) | Moderate | N/A (water) |
| Radar | Radio (mm–cm) | Long (km) | Low–Med | Excellent |
| LiDAR | Laser (near-IR) | Short–Med | Very high | Poor (fog/rain) |

## 1.4 Applications

Ranging sensors are found wherever a machine needs to *perceive* its physical environment in 3D. In some applications a *single* sensor type is sufficient; in others, *combining* several sensors is necessary because each has blind spots the others cover.

**Autonomous car navigation** — sensor combination (slide 7):

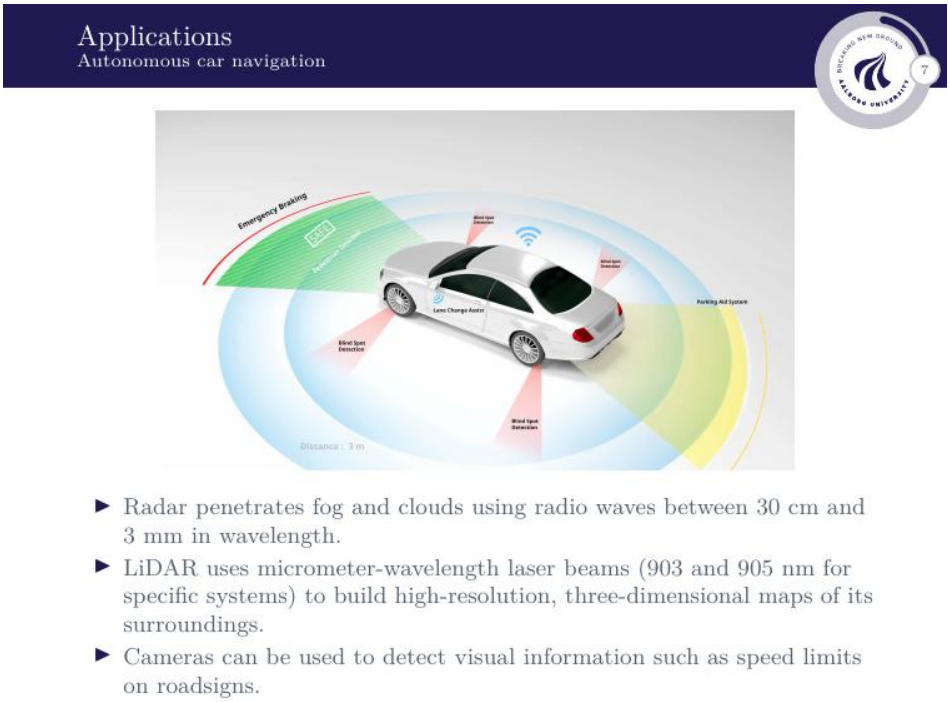


Figure 5: Three complementary sensors on an autonomous vehicle, each covering what the others cannot.

This is the clearest example of *why* sensors are combined:

- **Radar alone** can detect that *something* is ahead and how fast it is approaching (radial velocity via Doppler), and it works in fog, rain, and darkness. But its spatial resolution is too low to tell whether that something is a pedestrian, a parked car, or a road sign.
- **LiDAR alone** builds a precise 3D point cloud of the surroundings (sub-centimetre resolution) and can identify shapes of objects well. But it fails in heavy rain or fog because water droplets scatter the laser, and it cannot read text or colour (speed-limit signs, traffic lights).
- **Camera alone** reads colour, text, and rich visual detail (signs, lane markings, traffic lights), but gives no direct range measurement and struggles in darkness or glare.

Used *together*: radar handles adverse weather and provides velocity; LiDAR maps the 3D geometry of the scene in good conditions; the camera reads visual information no ranging sensor can capture. Each sensor fills in the gaps of the others — this is called **sensor fusion**.

**Underwater echo-location** — sonar alone (slide 8):

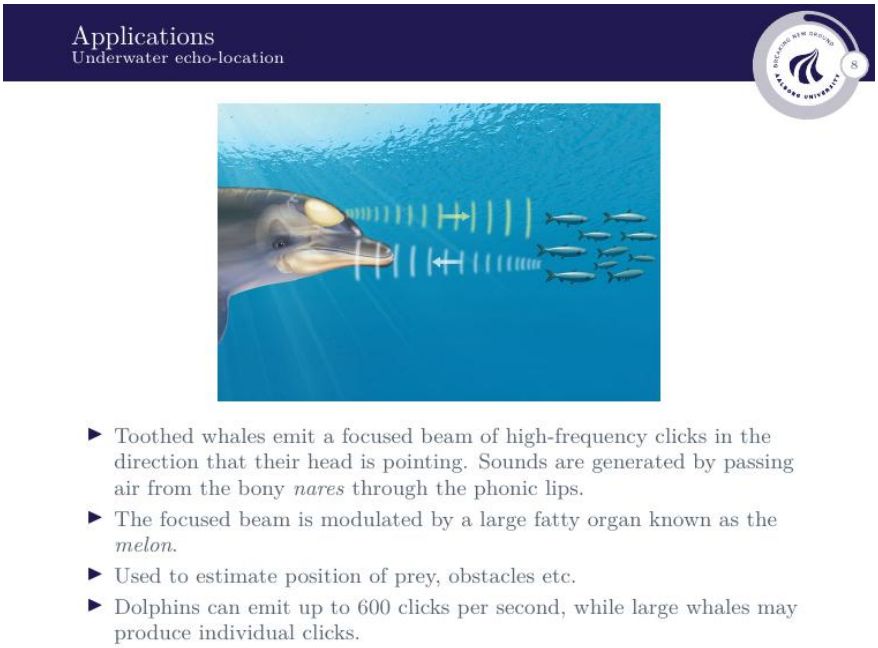


Figure 6: A dolphin using biological active sonar. Sound pulses emitted from the head reflect off prey; the returning echoes are used to estimate distance and direction. No combination needed — sonar is the only practical ranging method underwater, since light and radio waves are absorbed within metres.

Here a *single* sensor type is used because the medium (water) rules out all alternatives: radio waves are absorbed almost immediately underwater, and visible light scatters strongly beyond a few metres. Sound, however, propagates very efficiently in water. Active sonar alone is therefore sufficient to sense range and direction to targets.

**Geo-surveying** — LiDAR alone (slide 9):

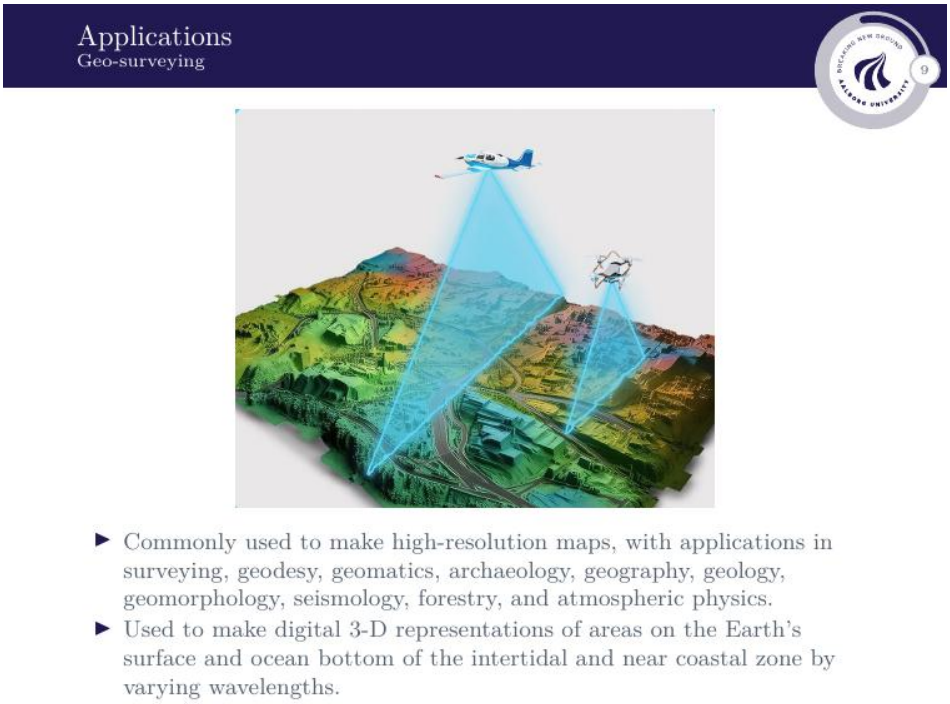


Figure 7: Airborne LiDAR geo-survey. The aircraft sweeps a laser beam downward; each reflected pulse gives one 3D ground point. The result is a high-resolution digital terrain model used in surveying, archaeology, geology, forestry, and atmospheric physics.

In open-air surveying from an aircraft, LiDAR alone is sufficient: the goal is purely to measure the 3D geometry of the terrain below with the highest possible resolution. Radar would give longer range but far lower spatial resolution, and a camera gives no direct range. LiDAR's ability to penetrate gaps in tree canopy and return *multiple echoes* (canopy return, lower canopy, ground return — see the waveform model in Section 3) makes it uniquely suited to this task.

## 1.5 Essential Components of a LiDAR Sensor

Before diving into the measurement physics, it helps to understand the *hardware* that makes LiDAR work.

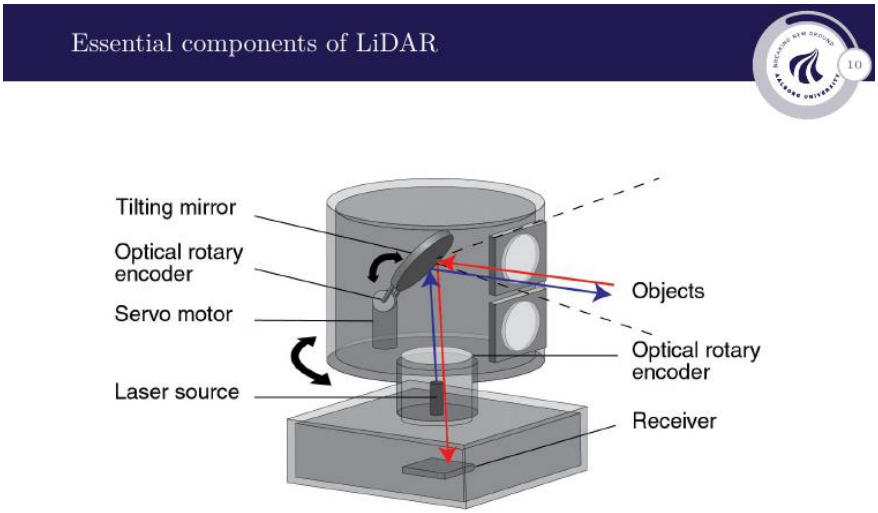


Figure 8: Cut-away of a rotating LiDAR head showing the five key components: (1) **Laser source** — emits the outgoing pulse. (2) **Tilting mirror** — steers the beam vertically. (3) **Servo motor** — rotates the entire head horizontally for full 360° azimuth coverage. (4) **Optical rotary encoders** — record the exact azimuth and elevation angles at the moment of each pulse, so every returned distance can be assigned its correct angular direction. (5) **Receiver** — captures the returning laser pulse.

- The **laser source** emits a short, powerful pulse of coherent light. The pulse duration directly affects range resolution (shorter pulse ⇒ finer resolution — see Section 2).
- The **tilting mirror** + **servo motor** combination allows the sensor to build a full 3D scan: the motor rotates the head through 360° in azimuth while the mirror tilts through a vertical field of view.
- The **optical rotary encoders** are critical for accuracy — without knowing the exact pointing angle at the moment of each pulse, the 3D coordinates of the return point cannot be computed.
- The **receiver** (photodetector) converts the returning photons back into an electrical signal. The time between transmit and receive is measured to give the distance.

**Section 1 — Key Takeaways**

- Ranging sensors measure distance by sending a wave and timing (or phase-comparing) the return. The three main families are **Sonar** (sound), **Radar** (radio waves), and **LiDAR** (laser light).
- **Sonar** is optimal underwater; active mode measures range, passive mode only

listens.

- **Radar** has the longest range and works in all weather, but lower spatial resolution. It can also measure radial velocity via the Doppler effect.
- **LiDAR** gives the highest spatial resolution (sub-centimetre point clouds) but requires clear air. It is rapidly becoming the dominant sensor in engineering applications (robotics, autonomous vehicles, mapping).
- A rotating LiDAR head combines a laser source, steering mirror, servo motor, rotary encoders (for angle), and a receiver. The encoders are what turn a 1D range into a 3D point.

# Section 2 Measurement Principles: Pulsed & Phased LiDAR

Now that we know *what* a LiDAR is made of, we look at *how* it actually measures distance. There are two fundamentally different approaches: **pulsed** LiDAR, which uses time of flight, and **phased** (continuous-wave) LiDAR, which uses the phase of a modulated wave. Both ultimately rely on the finite speed of light, but they extract the distance in different ways and come with different trade-offs in range and precision.

## 2.1 Pulsed LiDAR — Time of Flight

The simplest and most common mode. The laser fires a single short pulse ($< 50\,\text{ns}$), and a timer starts. When the reflected echo arrives at the receiver, the timer stops. The round-trip time $t$ gives the distance directly.

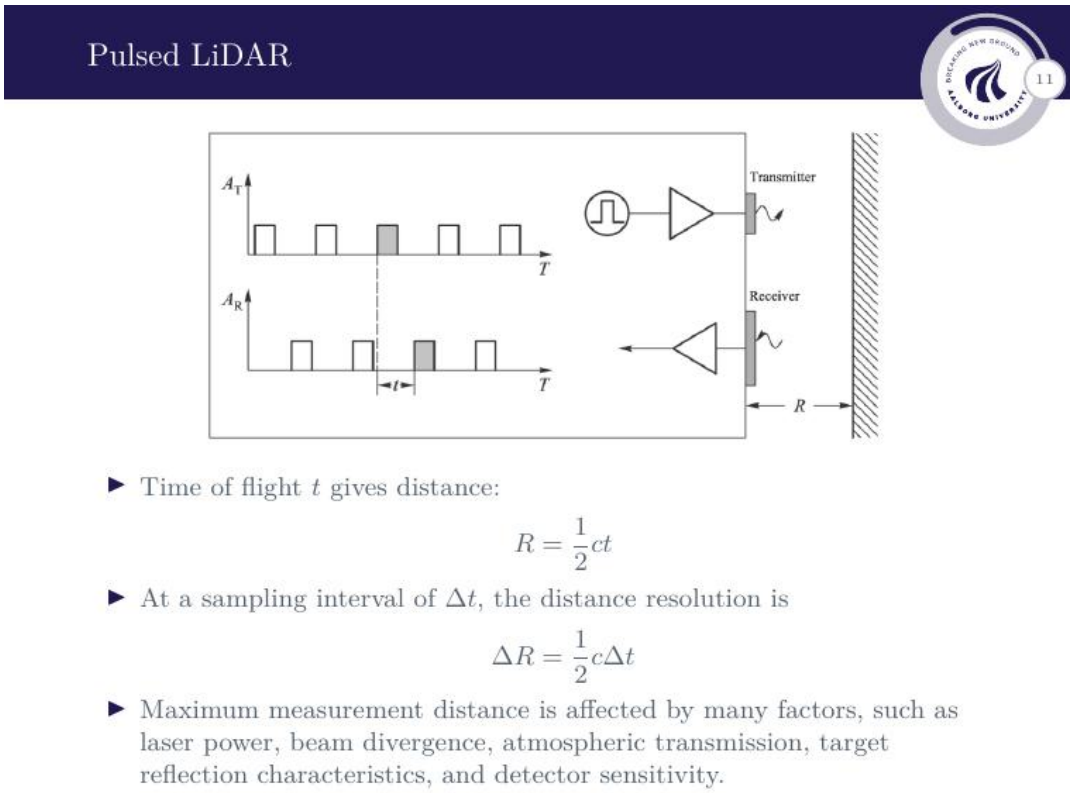


Figure 9: Pulsed LiDAR timing diagram. Top: the transmitted pulse train with period $T$ and amplitude $A_t$. Bottom: the received echo train, delayed by the round-trip time $t$ and reduced in amplitude $A_R$. Right: physical setup — the transmitter fires toward a surface at distance $R$; the receiver captures the return.

**Range from time of flight:**

The pulse travels to the target and back, covering a total distance of $2R$ at the speed of light $c$. Solving for $R$:

$$R = \frac{1}{2}\,c\,t$$

where $t$ is the measured round-trip time and $c \approx 3 \times 10^8\,\mathrm{m/s}$. The factor $\frac{1}{2}$ accounts for the *two-way* trip — the pulse must travel to the target *and* back.

**Distance resolution from sampling interval:**

In practice the timer is a digital clock with a fixed sampling interval $\Delta t$ (the smallest time step it can resolve). This limits how finely the range can be measured. The *distance resolution* — the smallest difference in range that can be distinguished — is:

$$\Delta R = \frac{1}{2}\, c\, \Delta t$$

A shorter clock tick $\Delta t$ gives finer range resolution. For example, $\Delta t = 1\,\mathrm{ns}$ gives $\Delta R = 15\,\mathrm{cm}$.

**What limits the maximum measurement distance?**

The formula $R = \frac{1}{2}ct$ gives range from time alone, but the sensor can only detect a return if the echo is strong enough to register above the noise floor of the receiver. The maximum range is therefore affected by:

- **Laser power** — more powerful pulse → stronger echo at long range.
- **Beam divergence** — a wider beam spreads energy over a larger footprint at distance, reducing the power density hitting the target and the fraction reflected back.
- **Atmospheric transmission** — air, water vapour, dust, and fog absorb and scatter the beam, attenuating it over distance.
- **Target reflectance** — a dark or rough surface absorbs more of the pulse; a bright or smooth surface reflects more back.
- **Detector sensitivity** — a more sensitive photodetector can register weaker returns.

All five factors feed into the *LiDAR range equation* (the scattering model derived in Section 3).

## 2.2 Phased LiDAR — Continuous Wave with Phase Comparison

Instead of firing a short pulse, phased (or phase-based) LiDAR emits a *continuous* laser beam whose *intensity is modulated* sinusoidally at a fixed frequency $f$. The receiver compares the *phase* of the returning wave with the phase of the transmitted wave. Because the light travels a round-trip distance $2R$ at speed $c$, the returning wave is delayed by time $t = 2R/c$, which appears as a phase shift $\phi$.

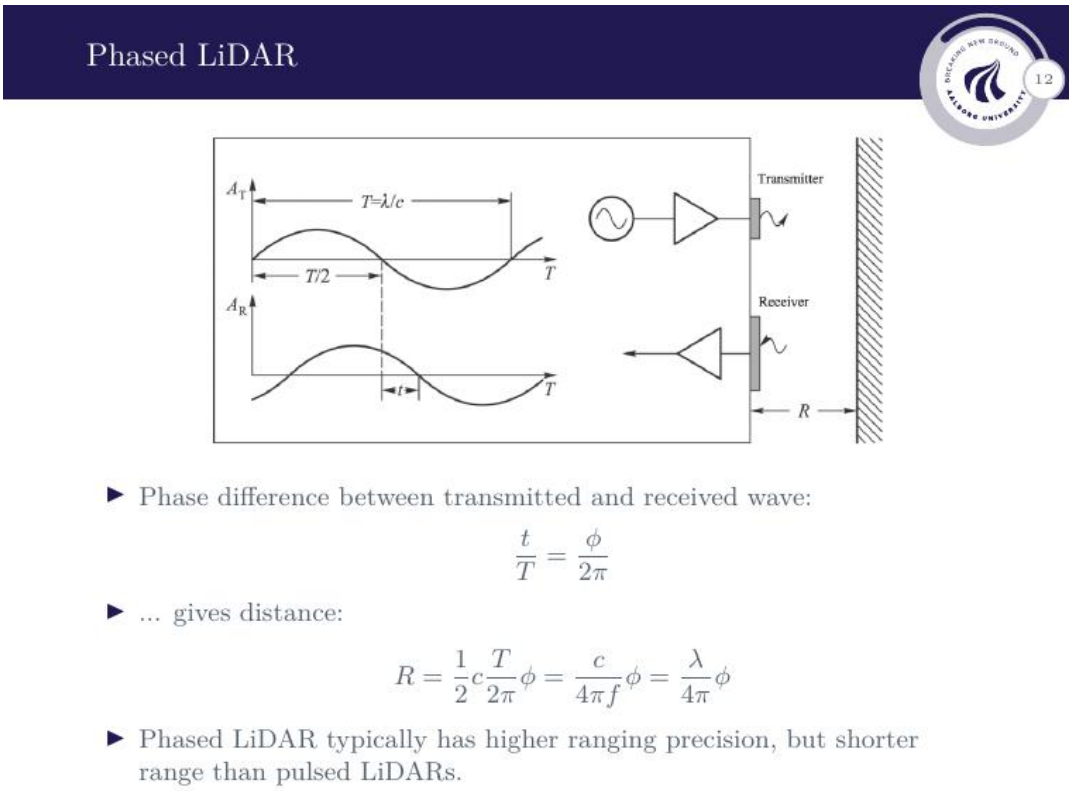


Figure 10: Phased LiDAR signals. Top: the transmitted sinusoidal wave with period $T = \lambda/c$ and amplitude $A_t$. Bottom: the received wave, shifted in phase by $\phi$ relative to the transmitted wave. The half-period $T/2$ corresponds to a round trip of one full wavelength $\lambda$.

**Phase difference to range:**

The wave completes one full cycle ($2\pi$ radians) in one period $T$. The received signal is delayed by time $t$, which is a fraction $t/T$ of one full cycle, corresponding to a phase shift $\phi$:

$$\frac{t}{T} = \frac{\phi}{2\pi} \quad \Longrightarrow \quad t = \frac{T\,\phi}{2\pi}$$

Substituting into $R = \frac{1}{2}ct$ and using $T = 1/f = \lambda/c$:

$$R = \frac{1}{2}\, c\, \frac{T\,\phi}{2\pi} = \frac{c}{4\pi f}\, \phi = \frac{\lambda}{4\pi}\, \phi$$

where $\lambda = c/f$ is the modulation wavelength (not the optical wavelength of the laser itself, but the wavelength of the *intensity modulation*).

**Intuition for the phase formula.**

Think of the modulated intensity as a ruler laid along the beam. One full wavelength $\lambda$ of the modulation corresponds to a round-trip distance of $\lambda$ (i.e. a one-way range of $\lambda/2$). Measuring the phase $\phi$ tells you *where along that ruler* the target sits:

$$R = \frac{\lambda}{4\pi}\, \phi \qquad \text{(range increases linearly with measured phase)}$$

A full phase cycle $\phi = 2\pi$ gives $R = \lambda/2$ — the maximum *unambiguous* range. Beyond this, the phase wraps around and the system cannot tell whether $\phi$ corresponds to one range or a range one modulation wavelength further away. This is the fundamental range ambiguity of phased LiDAR.

| | **Pulsed LiDAR** | **Phased LiDAR** |
|---|---|---|
| Range from | Time of flight $t$ | Phase shift $\phi$ |
| Formula | $R = \frac{1}{2}ct$ | $R = \frac{\lambda}{4\pi}\phi$ |
| Range limit | Laser power / atmosphere | Modulation wavelength ($\lambda/2$) |
| Typical range | Long ($> 100\,\text{m}$) | Shorter ($\sim$1–100 m) |
| Precision | Moderate | Higher |

**Why phased LiDAR is more precise.** Pulsed LiDAR measures a very short *time interval* (nanoseconds); even a 1 ns clock gives only $\Delta R = 15\,\text{cm}$ resolution. Phased LiDAR instead measures a *phase angle*, which electronic circuits (phase-locked loops, mixers) can resolve to fractions of a degree. Furthermore, the continuous wave *averages over many cycles*, suppressing noise the same way a longer integration time helps any measurement. The result: sub-millimetre precision is achievable with phased LiDAR at short ranges, whereas pulsed systems rarely do better than a few centimetres. The cost, as shown above, is the limited unambiguous range $\lambda/2$.

**Example calculation — phased LiDAR range**

Suppose the intensity modulation frequency is $f = 10\,\text{MHz}$. Then:

$$\lambda = \frac{c}{f} = \frac{3 \times 10^8}{10 \times 10^6} = 30\,\text{m} \qquad R_{\max} = \frac{\lambda}{2} = 15\,\text{m (unambiguous range)}$$

If the receiver measures a phase shift of $\phi = 120° = \frac{2\pi}{3}$ rad:

$$R = \frac{\lambda}{4\pi}\phi = \frac{30}{4\pi} \cdot \frac{2\pi}{3} = \frac{30}{6} = 5\,\text{m}$$

The target is 5 m away.

**The ambiguity problem — why the target must be closer than $\lambda/2$.**

You are right to flag this. The phase $\phi$ is always measured in the range $[0, 2\pi)$ — the receiver cannot count how many full cycles the wave has completed; it only sees *where in the current cycle* the return sits. This means that any target further than $\lambda/2$ produces a phase that *wraps around* and looks identical to a closer target.

*Concrete example:* take the same $\lambda = 30\,\text{m}$ system.

- **Target A** at $R = 5\,\text{m}$: $\phi = \frac{4\pi \times 5}{30} = \frac{2\pi}{3} \approx 120°$ $\quad \rightarrow$ measured as 120° ✓
- **Target B** at $R = 20\,\text{m}$: $\phi = \frac{4\pi \times 20}{30} = \frac{8\pi}{3} \approx 480°$ $\quad \rightarrow$ wraps: $480° - 360° = 120°$

Target A at 5 m and Target B at 20 m give **exactly the same measured phase of** 120°. The system cannot tell them apart.

The wrap occurs at every multiple of $\lambda/2 = 15\,\mathrm{m}$. So 5 m, 20 m, 35 m, ... are all indistinguishable.

**Design rule:** choose the modulation frequency $f$ so that $\lambda/2$ is *larger* than the furthest target you ever expect:

$$f < \frac{c}{2\,R_{\mathrm{max,\,expected}}}$$

Lower frequency → longer $\lambda$ → larger unambiguous range, but also coarser phase ruler → worse precision. This is the core trade-off of phased LiDAR.

*Note:* some systems use **two modulation frequencies** simultaneously. The beat between them effectively creates a much longer "virtual wavelength", extending the unambiguous range while keeping fine precision from the higher frequency — similar to a vernier scale.

## 2.3 Ranging Precision

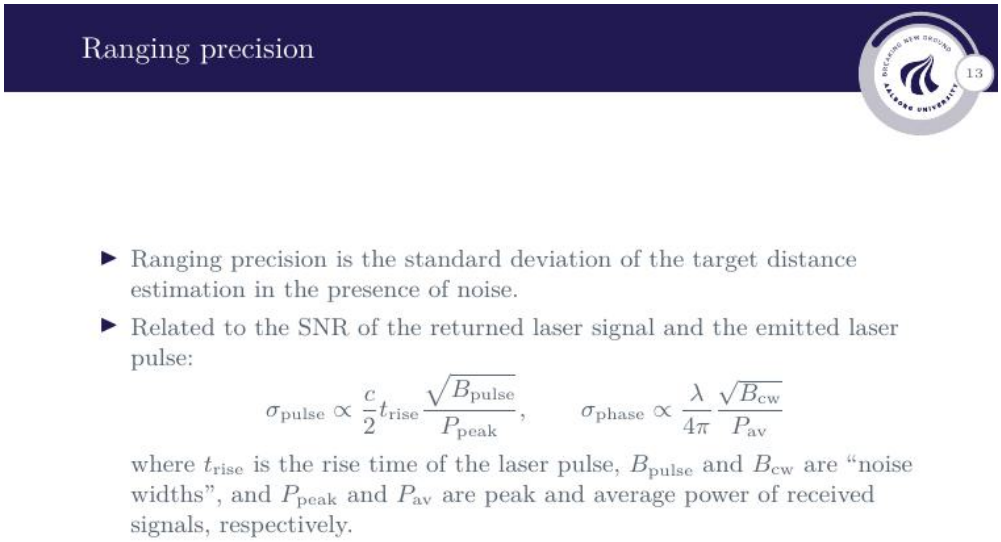


Figure 11: Ranging precision slide. The two formulas give the standard deviation of the range estimate for pulsed and phased LiDAR respectively, showing how noise bandwidth and received power govern measurement quality.

**Ranging precision** is defined as the *standard deviation* $\sigma$ of the distance estimate in the presence of noise. It is related to the **signal-to-noise ratio (SNR)** of the returned laser signal: a stronger return relative to noise gives a more precise distance estimate.

For the two LiDAR types, the precision scales as:

$$\sigma_{\text{pulse}} \propto \frac{c}{2} t_{\text{rise}} \sqrt{\frac{B_{\text{pulse}}}{P_{\text{peak}}}} \qquad \sigma_{\text{phase}} \propto \frac{\lambda}{4\pi} \sqrt{\frac{B_{\text{cw}}}{P_{\text{av}}}}$$

**Variable definitions**

| Symbol | Name | Meaning / effect on precision |
|---|---|---|
| $\sigma_{\text{pulse}}$ | Pulsed ranging std. dev. | Smaller = better precision |
| $\sigma_{\text{phase}}$ | Phased ranging std. dev. | Smaller = better precision |
| $c$ | Speed of light ($3 \times 10^8$ m/s) | Fixed constant |
| $t_{\text{rise}}$ | Rise time of laser pulse | Shorter pulse edge → sharper timing → better precision |
| $\lambda$ | Modulation wavelength | Shorter $\lambda$ → finer phase ruler → better precision |
| $B_{\text{pulse}}$ | Noise bandwidth (pulsed) | Wider bandwidth captures more noise → worse precision |
| $B_{\text{cw}}$ | Noise bandwidth (phased/CW) | Same as above: wider = noisier |
| $P_{\text{peak}}$ | Peak power of received pulse | More power → stronger signal → better precision |
| $P_{\text{av}}$ | Average power of received CW signal | More power → better precision |

**Understanding the key terms before the table**

**Rise time $t_{\text{rise}}$ — how sharp is the pulse edge?** The pulsed laser signal is not a sine wave — it is a short rectangular burst (on/off), like a camera flash. The rise time is simply how fast the leading edge switches from zero to full power. Any sharp edge in time is equivalent to having high-frequency content in the signal (Fourier analysis: a perfectly sharp step would require infinitely high frequencies). So:

$$\text{shorter } t_{\text{rise}} \longleftrightarrow \text{steeper edge} \longleftrightarrow \text{higher frequency content in pulse}$$

Because a blunt, slow-rising edge makes it hard to say exactly *when* the pulse arrived, shorter $t_{\text{rise}}$ gives sharper timing and better range precision.

**Noise bandwidth $B$ — how much noise does the receiver let in?** Every receiver circuit responds to a range of frequencies — its bandwidth. Noise (thermal, shot) is spread across *all* frequencies, so a wider bandwidth lets more noise through alongside the signal, degrading precision. $B_{\text{pulse}}$ and $B_{\text{cw}}$ are the same concept but very different values in practice:

- $B_{\text{pulse}}$ must be *wide* — a nanosecond pulse has frequency content up to ∼1 GHz, so the pulsed receiver needs GHz-range bandwidth to capture the sharp edges faithfully.

- $B_{\text{cw}}$ can be *narrow* — only a small band around the modulation frequency (e.g. 10 MHz) needs to pass; all other frequencies are noise and can be filtered out.

This is part of why phased LiDAR achieves better precision: its narrow $B_{\text{cw}}$ admits far less noise than the wide $B_{\text{pulse}}$ of a pulsed receiver, even at equal received power.

**The "ruler" — what physical distance corresponds to one unit of measurement?** Every measurement needs a scale. In each LiDAR type, one unit of the measured quantity (time tick or phase radian) maps to a certain number of metres:

- **Pulsed:** one clock tick $\Delta t = 1\,\text{ns} \rightarrow \frac{1}{2}c\,\Delta t = 0.15\,\text{m}$. So 15 cm is one graduation on the pulsed ruler. A shorter rise time shrinks this graduation.
- **Phased:** one radian of phase $\rightarrow \frac{\lambda}{4\pi}$ m. With $\lambda = 30\,\text{m}$ that is $\sim$2.4 m per radian — sounds coarse, but electronics can resolve 0.001 rad easily, giving $\sim$2.4 mm precision. A shorter modulation wavelength $\lambda$ makes the ruler finer.

The $\sigma$ formulas simply say: precision = (ruler graduation) $\times$ (noise-to-signal ratio).

The key insight is the **square-root SNR dependence**: precision scales as $\sqrt{B/P}$. To halve $\sigma$ (double the precision), you need to *quadruple* the received power, or quarter the noise bandwidth — improvements are expensive because of the square root.

Notice also the **structural similarity** between the two formulas:

- Pulsed: the "ruler" is set by $\frac{c}{2}\,t_{\text{rise}}$ — essentially the spatial length of the pulse edge in metres. The shorter and sharper the pulse edge, the finer the timing and the better the precision.
- Phased: the "ruler" is set by $\frac{\lambda}{4\pi}$ — one radian of phase in metres. A shorter modulation wavelength gives a finer ruler, hence better precision. This is why phased LiDAR typically outperforms pulsed LiDAR in precision at short ranges.

**Section 2 — Key Takeaways**

- **Pulsed LiDAR** fires a short burst and measures round-trip time: $R = \frac{1}{2}ct$. Distance resolution is set by the clock interval: $\Delta R = \frac{1}{2}c\,\Delta t$.
- **Phased LiDAR** modulates a continuous beam and measures the phase shift of the return: $R = \frac{\lambda}{4\pi}\phi$. Higher precision but limited unambiguous range ($\leq \lambda/2$).
- **Ranging precision** $\sigma \propto \sqrt{B/P}$: halving $\sigma$ requires quadrupling received power. Precision improves with sharper pulse edges (pulsed) or shorter modulation wavelength (phased).
- The **maximum range** is governed by physics beyond the timing formula: laser power, beam divergence, atmosphere, target reflectance, and detector sensitivity all play a role (covered fully in Section 3).

# Section 3 LiDAR Scattering & Waveform Model

We now know *how* LiDAR measures time or phase to get distance. But there is a deeper question: *how much* of the emitted laser power actually makes it back to the sensor? This matters because if the returned power is too weak the measurement fails entirely — range is not infinite. The **scattering model** traces the laser power step by step from transmitter to target to receiver, giving a formula for the received power $P_R$. The **waveform model** then explains what happens when a single pulse hits a complex target such as a tree, producing multiple overlapping returns.

## 3.1 The Four-Step Power Chain

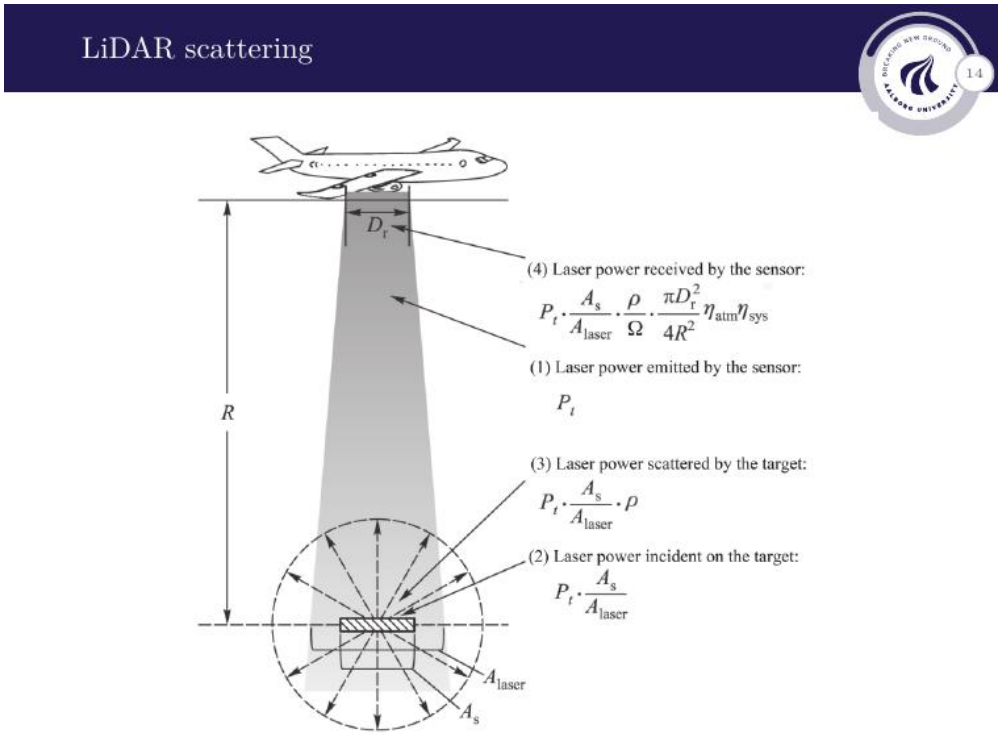


Figure 12: LiDAR scattering geometry. The laser illuminates a footprint of area $A_{\text{laser}}$ on the ground at range $R$. The target scatters power in all directions; only the fraction caught by the receiver aperture $A_r = \pi D_r^2/4$ returns to the sensor. The four numbered steps track the power budget from transmitter to receiver.

It helps to follow the power through four physical steps:

1. **Laser emits power $P_t$.** The transmitter sends out a pulse with total power $P_t$. This power spreads over a cone as it travels downward.

2. **Power incident on the target.** At range $R$, the beam illuminates a circular footprint of area $A_{\text{laser}}$ on the ground. $A_{\text{laser}}$ is the *total* area lit up by the beam — it can be large (a wide footprint at long range hits many things: ground, rocks, vegetation). The target itself only occupies part of that footprint; its *effective scattering*

*area* $A_S \leq A_{\text{laser}}$ is how much of the footprint it fills. A small rock sitting inside a large footprint has $A_S \ll A_{\text{laser}}$ — only a tiny fraction of the emitted power actually hits it. The ratio $A_S/A_{\text{laser}}$ is therefore the fraction of $P_t$ that reaches the target:

$$P_{\text{incident}} = P_t \cdot \frac{A_S}{A_{\text{laser}}}$$

3. **Target scatters power.** The *reflectance* $\rho$ ($0 \leq \rho \leq 1$) is primarily a **material property** — it describes what fraction of the incident light the surface reflects rather than absorbs. Dark asphalt has $\rho \approx 0.05$; white paint has $\rho \approx 0.9$. The shape and roughness of the surface determine the *direction* the light is scattered (captured by $\Omega$), but $\rho$ itself is the material's reflective fraction — the two effects are kept separate in the model. The scattered power is:

$$P_S = P_t \cdot \frac{A_S}{A_{\text{laser}}} \cdot \rho$$

4. **Receiver captures a fraction of the scattered power.** $\Omega$ is the *full cone of directions* the target scatters light into — it has nothing to do with where the sensor is. Three cases illustrate the range:

   - **Mirror / retroreflector** — reflects nearly all light back in one narrow direction. $\Omega$ is tiny; most scattered power lands on the sensor → very strong return. This is why road signs and survey targets coated with retroreflective material give extremely bright LiDAR returns.
   - **Rough / diffuse surface** (soil, bark, asphalt) — scatters light roughly equally in all directions of the upper hemisphere. $\Omega \approx 2\pi$ steradians. The sensor intercepts only a tiny slice → weak return.
   - **Most real surfaces** sit somewhere between: a preferred specular peak plus a broad diffuse component.

   The formula captures this directly: received fraction $= A_r/(\Omega R^2)$. Smaller $\Omega$ (more directed scatter) → larger fraction back to the sensor.

   The scattered power spreads over solid angle $\Omega$ at range $R$, covering an area of $\Omega R^2$. The receiver lens has diameter $D_r$, so its collecting area is:

$$A_r = \frac{\pi D_r^2}{4} \quad \text{(area of a circle with diameter } D_r\text{)}$$

   This is simply $\pi r^2$ with $r = D_r/2$ — a bigger lens intercepts more of the scattered sphere. The fraction captured is $A_r/(\Omega R^2)$.

   The two efficiency factors $\eta_{\text{atm}}$ and $\eta_{\text{sys}}$ cover losses on **both the outgoing and return paths**, not just the return. They appear once at the end because $P_t$ is defined as the power at the laser output; all subsequent losses — atmosphere on the way *to* the target, atmosphere on the way *back*, plus sensor optics and electronics — are multiplied in together as a combined round-trip efficiency. (If you wanted to split them: $\eta_{\text{atm}} = \eta_{\text{atm,go}} \times \eta_{\text{atm,return}}$, but in practice a single factor is measured and used.) The received power is:

$$P_R = P_t \cdot \frac{A_S}{A_{\text{laser}}} \cdot \rho \cdot \frac{1}{\Omega R^2} \cdot \frac{\pi D_r^2}{4} \cdot \eta_{\text{atm}}\, \eta_{\text{sys}}$$

## 3.2 Deriving the Full Scattering Model

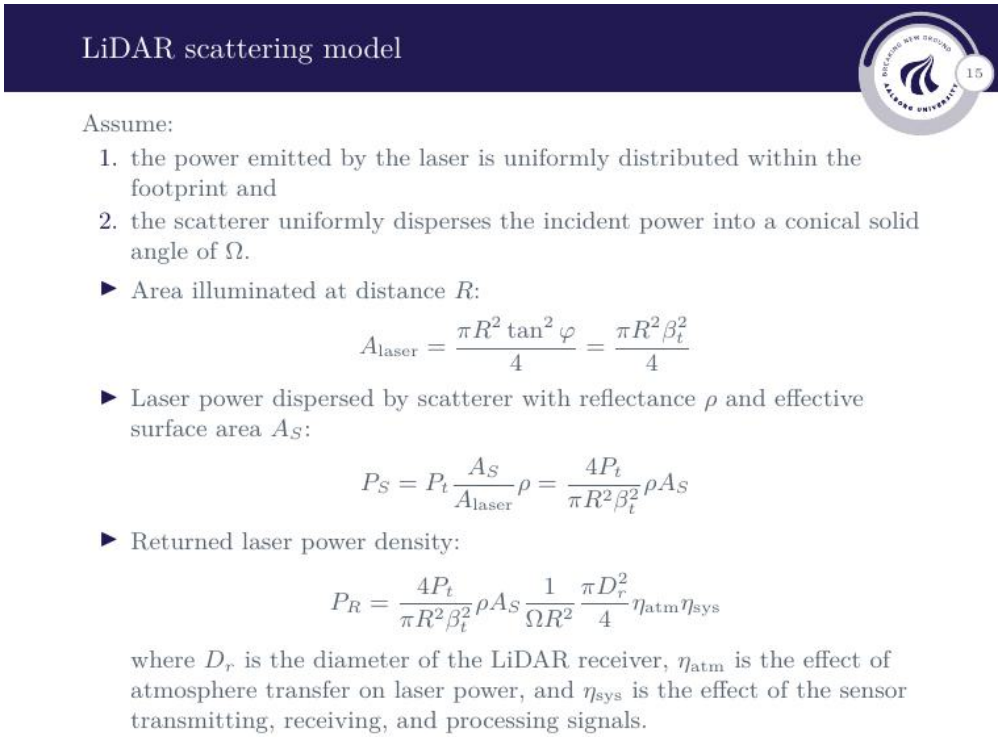


Figure 13: LiDAR scattering model with labelled geometry. $\beta_t$ is the beam divergence half-angle; the illuminated footprint grows as $R^2$. The scatterer disperses power uniformly into solid angle $\Omega$.

Two assumptions are made to make the model tractable:

1. The emitted laser power is **uniformly distributed** within the footprint (flat-top beam profile).
2. The scatterer **uniformly disperses** the incident power into a conical solid angle $\Omega$ — a so-called **Lambertian** surface model.

**Assumption 2 is a simplification — real surfaces do not scatter uniformly.**

The Lambertian assumption says power is spread *evenly* inside $\Omega$. Real surfaces do not do this. Ridges, bumps, and surface texture mean some scatter directions get more energy than others. Most surfaces also have a **specular component** — a stronger peak of reflection in the mirror direction on top of the broad diffuse scatter. The angle the beam hits the surface at further changes the pattern.

The full description of how a real surface scatters is called the **BRDF** (Bidirectional Reflectance Distribution Function), which gives a different value for every incoming/outgoing angle combination.

The model uses Lambertian scattering for three practical reasons:

- The BRDF of every target in the scene is unknown in advance.

- Uniform $\Omega$ makes the maths tractable and gives one closed-form formula for $P_R$.
- For most rough natural surfaces (soil, vegetation, asphalt) the non-uniformities average out over many measurements, so the Lambertian model is a reasonable mean estimate.

$\Omega$ therefore acts as a single model parameter that lumps all unknown directional scattering behaviour into one number. The formula gives a useful engineering estimate, not an exact value.

**Step 1 — How large is the illuminated footprint at range $R$?**

The laser beam is not a perfectly parallel ray — it spreads slightly as it travels, forming a cone. The **half-angle** $\varphi$ is the angle between the centre axis of the beam and its outer edge, i.e. half the total opening angle of the cone. Think of a flashlight: a wide half-angle gives a broad circle of light on the wall; a narrow half-angle gives a tight spot.

At range $R$, the beam has spread to a circular footprint of radius $r = R\tan\varphi \approx R\beta_t/2$, so its area is:

$$A_{\text{laser}} = \pi r^2 = \frac{\pi R^2 \tan^2\varphi}{4} = \frac{\pi R^2 \beta_t^2}{4}$$

*What this tells you:* $A_{\text{laser}}$ is what we called the footprint area back in Step 2 of the four-step chain. Now we have an explicit formula for it. Because $A_{\text{laser}} \propto R^2$, doubling the range quadruples the footprint — the same total power $P_t$ is spread over four times the area, so the power *density* on the ground drops to one quarter. A narrow beam (small $\beta_t$) keeps the footprint small and the power density high even at long range.

The beam itself is **3D** — a cone expanding in all directions as it travels. The footprint it makes on the ground is **2D** — a flat circle. The formula $\pi R^2\beta_t^2/4$ assumes the *same* half-angle $\beta_t$ in every direction around the beam axis, i.e. the cone is perfectly circular in cross-section. If the divergence were different in the two perpendicular directions ($\beta_x \neq \beta_y$), the footprint would instead be an **ellipse** with area $\pi R^2\beta_x\beta_y/4$. Most LiDAR systems are designed for a circular beam, so $\beta_x = \beta_y = \beta_t$ is a reasonable assumption.

**Step 2 — How much power reaches the sensor after scattering?**

We now substitute the explicit $A_{\text{laser}}$ from Step 1 into the scattered-power formula from Step 3 of the four-step chain ($P_S = P_t \cdot \frac{A_S}{A_{\text{laser}}} \cdot \rho$):

$$P_S = P_t \cdot \frac{A_S}{\frac{\pi R^2\beta_t^2}{4}} \cdot \rho = \frac{4\,P_t}{\pi R^2\beta_t^2}\,\rho\,A_S$$

*What this tells you:* scattered power already falls off as $1/R^2$ — the bigger footprint dilutes the power hitting the target. This uses the Lambertian model from assumption 2: the target disperses the incident power uniformly across $\Omega$, so the fraction that eventually returns depends purely on $\rho$, $A_S$, and how large $A_{\text{laser}}$ is at that range.

**Step 3 — What power finally arrives at the receiver?**

We now take the full expression from Step 4 of the four-step chain and substitute $A_{\text{laser}}$ everywhere:

$$\boxed{P_R = \frac{4\,P_t}{\pi R^2 \beta_t^2} \cdot \rho A_S \cdot \frac{1}{\Omega R^2} \cdot \frac{\pi D_r^2}{4} \cdot \eta_{\text{atm}}\,\eta_{\text{sys}}}$$

*What this tells you:* this is the complete **LiDAR range equation**. The two $1/R^2$ factors combine to give a $1/R^4$ penalty — doubling the range reduces returned power by 16×. The formula also makes clear which design choices help: a narrower beam ($\beta_t$ smaller), a larger receiver lens ($D_r$ larger), a more reflective or larger target ($\rho A_S$ larger), and low atmospheric loss ($\eta_{\text{atm}}$ close to 1) all directly increase $P_R$. This is exactly what limits the maximum range mentioned back in Section 2 (the five limiting factors in the pulsed LiDAR notebox) — they all appear here explicitly.

**Variable definitions**

| Symbol | Name | Meaning |
|---|---|---|
| $P_t$ | Transmitted power | Total laser pulse power emitted |
| $R$ | Range | Distance from sensor to target |
| $\beta_t$ | Beam divergence | Half-angle of the emitted cone; wider = larger footprint |
| $A_{\text{laser}}$ | Footprint area | Illuminated area at range $R$; grows as $R^2$ |
| $A_S$ | Scatterer area | Effective reflecting area of the target |
| $\rho$ | Reflectance | Fraction of incident power reflected (0–1); dark surfaces $\approx 0$, bright $\approx 1$ |
| $\Omega$ | Solid angle | Cone into which the target disperses the power |
| $D_r$ | Receiver diameter | Aperture of the sensor lens; larger = captures more |
| $\eta_{\text{atm}}$ | Atmospheric efficiency | Loss due to absorption and scattering in air ($\leq 1$) |
| $\eta_{\text{sys}}$ | System efficiency | Loss in the sensor's own optics and electronics ($\leq 1$) |

**Key observation: the $R^4$ penalty.** Notice that $P_R \propto 1/R^4$ — the received power drops as the *fourth power* of range. The $1/R^2$ comes from the footprint growing (less power hits the target), and another $1/R^2$ comes from the scattered power spreading over the hemisphere before the receiver catches its small slice. Doubling the range reduces returned power by a factor of 16, which is why long-range LiDAR requires powerful lasers and sensitive detectors.

### 3.3 Multiple Returns — The "But. . ." Slide

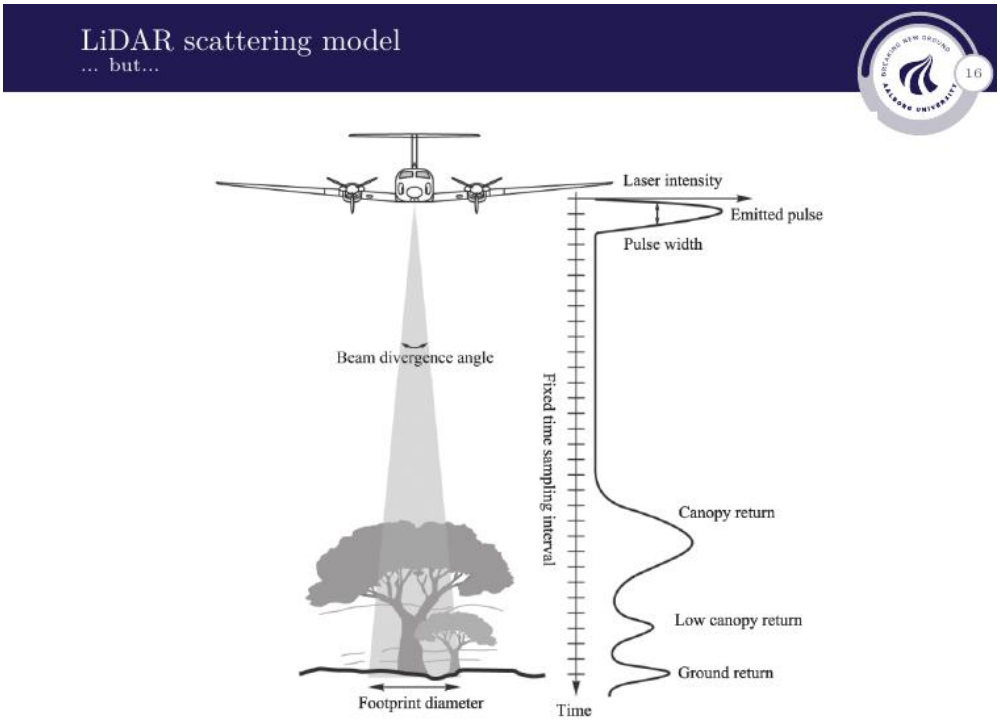


Figure 14: A single LiDAR pulse hitting a tree. The beam illuminates a footprint that includes both foliage and ground. The pulse partially reflects off the tree canopy (first return), travels further to lower branches (second return), and finally reflects off the ground (ground return). The receiver records a time-series of overlapping pulses rather than a single clean echo.

The scattering model above assumed a single flat target at range $R$. In practice, a single pulse often illuminates a *complex* scene — a tree, a building with windows, or uneven terrain. Different parts of the scene are at different ranges, so the pulse reflects back at different times, and the receiver records a stretched, multi-peaked waveform instead of a single clean echo.

As shown in the figure, one pulse hitting a tree produces three peaks in the received waveform:

- A **canopy return** — the first reflection from the top of the tree. Arrives earliest (shortest range).
- A **low canopy / branch return** — a weaker echo from lower branches that the pulse partially penetrated.
- A **ground return** — the remaining pulse energy that passed through all gaps in the foliage and reflected off the ground. Arrives last (longest range).

Each peak in the waveform has a **height** (amplitude) and a **width**. Here is what each tells you:

**Peak height — how much power came back from that surface.** A taller peak

means more power returned. But the height is governed by the full scattering formula $P_R \propto \rho A_S / R^4$, so a shorter peak could mean:

- A weakly-reflecting surface (small $\rho$), *or*
- A small surface area (small $A_S$), *or*
- Simply a surface **farther away** (large $R$, $R^4$ scaling).

In the tree example: the ground return peak is typically the weakest of the three, but this does **not** mean the ground is a poor reflector. The ground is simply the furthest surface — the pulse has already lost energy passing through canopy and branches, and the remaining path is longest. You cannot conclude anything about reflectance from height alone without first accounting for range.

**Peak width — what the surface looks like.** The emitted pulse has a finite duration, so every return has a minimum width. Beyond that, the echo is broadened by:

- **Surface roughness** — height variations within the footprint mean different parts reflect at slightly different ranges, smearing the echo in time.
- **Surface tilt** — a tilted surface has one edge closer than the other, broadening the return.
- **Depth extent** — a thick layer (dense foliage) reflects from both front and back, producing a wide merged peak.

In the tree example: the canopy return is often broad (the canopy has depth and rough texture); the ground return is typically narrow and sharp (flat, smooth ground reflects cleanly).

**You need timing, height, and width together.** The peak arrival time gives range; height corrected for that range gives reflectance; width gives surface character. None of the three is meaningful in isolation — a low, broad peak near the sensor could be a rough nearby surface, while an equally low but narrow peak far away is likely a smooth flat surface simply attenuated by $R^4$. This is why full-waveform LiDAR records the *entire* time-series rather than just the first return time.

## 3.4 LiDAR Waveform Model

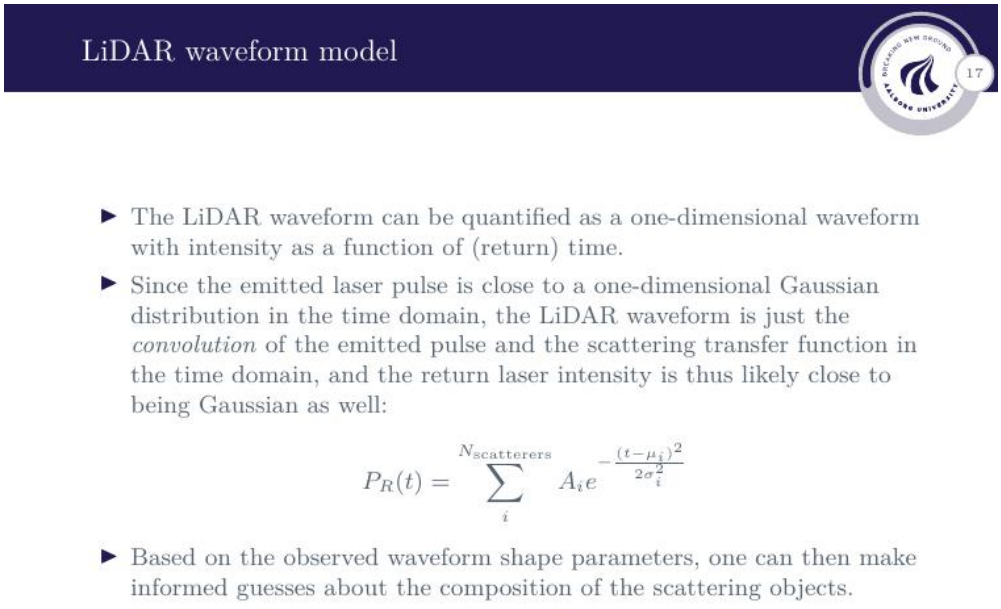


Figure 15: LiDAR waveform model. The emitted pulse is approximately Gaussian in time. Convolution with the scattering transfer function (one Gaussian per scatterer) produces the full received waveform, which is a sum of Gaussians — one per distinct reflecting surface.

This model applies to **pulsed** LiDAR — the sensor fires discrete pulses and records the returned power as a function of time. Phased LiDAR instead compares the phase of a continuous wave and does not produce a time-series waveform of this kind.

**Why the emitted pulse is Gaussian in time.** When the laser fires, the amplitude does not switch on and off instantaneously — it ramps up, reaches a peak, and ramps back down. The rise and fall are smooth, making the pulse shape approximately **Gaussian**: a bell curve in time with the peak at the centre of the pulse duration. There is no perfectly sharp pulse in practice; finite rise time (recall Section 2) is exactly this gradual ramp.

**Why each surface return is also Gaussian.** When this Gaussian pulse hits a single flat surface at range $R_i$, the surface reflects a delayed, scaled copy of the incoming pulse back toward the sensor. The copy arrives delayed by $2R_i/c$ and scaled by the surface's reflectance and geometry — but its *shape* is unchanged. Since the incoming pulse is Gaussian, the reflected copy is also Gaussian.

**Convolution — why we use that word.** Mathematically, the received waveform is the *convolution* of the emitted pulse with the *scattering transfer function*. The scattering transfer function describes the scene: it has a spike at each time $\mu_i = 2R_i/c$ with height $A_i$ for each reflecting surface. Convolving the emitted Gaussian with each spike shifts and scales a copy of the Gaussian to that time — one copy per surface. The key property that makes the model clean: **Gaussian** $*$ **Gaussian** $=$ **Gaussian**, so each return stays

Gaussian regardless of how many surfaces there are.

**The full waveform is the sum of all these Gaussians.** With $N$ reflecting surfaces, the receiver sees all their reflected copies arriving on top of each other. The total received power at any time $t$ is just their sum:

$$P_R(t) = \sum_{i=1}^{N_{\text{scatterers}}} A_i \exp\left(-\frac{(t-\mu_i)^2}{2\sigma_i^2}\right)$$

Each term in the sum is one Gaussian peak: one surface, one range, one echo. The full waveform is the overlay of all of them.

**Variable definitions and what each tells you**

| Symbol | Name | Meaning |
|---|---|---|
| $P_R(t)$ | Received power waveform | Full time-series of returned laser power at the receiver |
| $N_{\text{scatterers}}$ | Number of scatterers | How many distinct reflecting surfaces the pulse hit |
| $A_i$ | Amplitude of return $i$ | How strongly surface $i$ reflected; related to $\rho$ and $A_S$ |
| $\mu_i$ | Mean (centre time) of return $i$ | Arrival time of the $i$-th echo → range via $R_i = \frac{1}{2}c\mu_i$ |
| $\sigma_i$ | Width of return $i$ | How spread-out the echo is; affected by pulse width, surface roughness, and slope of the target |

By fitting this model to the measured waveform you recover $A_i$, $\mu_i$, $\sigma_i$ for every peak simultaneously — range, reflectance, and surface character from a single shot. See Section 3.3 for what each parameter physically means in the context of the canopy example.

**Section 3 — Key Takeaways**

- Received power follows a **four-step chain**: emit → illuminate footprint → scatter at target → receiver catches a fraction.
- The full formula is $P_R \propto \rho A_S/(\beta_t^2 R^4)$ — received power drops as $R^4$. Doubling range means 16× less power returned.
- **Footprint grows as $R^2$**: beam divergence $\beta_t$ determines how quickly the illuminated area expands with range.
- A single pulse hitting a tree (or any layered scene) produces **multiple returns** at different times — canopy, branches, ground. These appear as separate Gaussian peaks in the waveform.
- The received waveform is modelled as a **sum of Gaussians**: $P_R(t) = \sum_i A_i \exp(-(t-$

$\mu_i)^2/2\sigma_i^2\big)$. Fitting this model extracts range, reflectance, and surface roughness for each scatterer simultaneously.

# Section 4 Point Cloud Data & Real Hardware

The previous sections explained how LiDAR measures range and how the returned power is modelled. The output of a LiDAR sensor is not a single distance — it is a **point cloud**: a collection of thousands or millions of 3D points, each carrying its own position, timestamp, and intensity value. This section covers what a point cloud actually contains, how it is stored, and what a real-world rotating LiDAR system looks like in practice.

## 4.1 What a Point Cloud Contains

A **point cloud** is simply a large collection of individual 3D points, where each point represents one surface location that was measured by the sensor. Think of it as a scatter plot in 3D space: the sensor fires thousands of laser pulses per second, each returning a range and direction, and every single measurement becomes one dot in the cloud. Enough dots together and the shape of the whole environment emerges — walls, ground, trees, vehicles — even though there is no continuous surface, just a dense set of discrete points.

The point cloud is **not stored on the sensor itself** — the sensor has no meaningful on-board memory. Instead it streams the raw measurements in real time to a connected computer (in the VLP-16 case, via Ethernet UDP packets, as covered in Section 4.2). The computer then assembles the stream into a point cloud file on disk in a standard format such as LAS or LAZ.

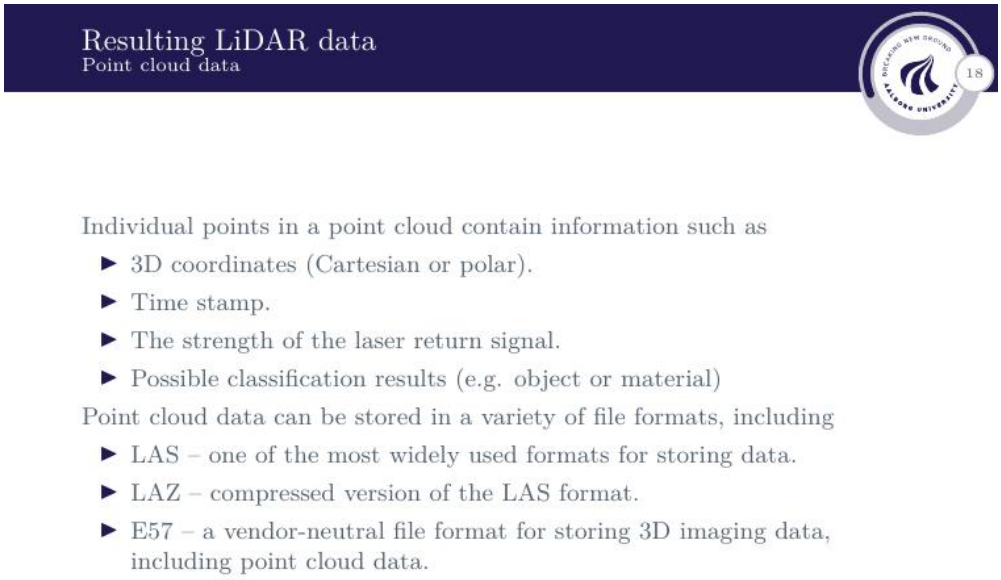


Figure 16: Point cloud data structure. Each individual point in the cloud carries several pieces of information beyond just its 3D position.

Each individual point in a point cloud contains:

- **3D coordinates** — either Cartesian $(x, y, z)$ or polar $(R, \alpha, \theta)$ (range, azimuth, elevation). In a rotating LiDAR the sensor natively measures polar coordinates; these are then converted to Cartesian using the rotary encoder angles recorded at the moment of each shot.
- **Timestamp** — the exact time the pulse was fired. This is critical for moving platforms (robots, cars, aircraft): without a timestamp, points recorded at different moments during motion cannot be correctly placed in a global coordinate frame.
- **Return signal strength (intensity)** — the amplitude $A_i$ of the returned pulse, as discussed in Section 3. This is the "colour" value of the point: brighter = more reflective surface. See the coloured-intensity image below (slide 19).
- **Classification results** — some systems attach a label to each point indicating what type of object it likely belongs to (e.g. ground, vegetation, building, vehicle). This is the output of a post-processing step, not a raw sensor measurement.

**Point cloud file formats**

Point cloud data can be stored in several standard formats:

- **LAS** — the most widely used industry standard for storing LiDAR point clouds. Stores 3D coordinates, intensity, timestamp, classification, and other attributes in a compact binary format.
- **LAZ** — a lossless compressed version of LAS. Typically 5–10× smaller file size with no data loss; the standard choice for archiving and transfer.
- **E57** — a vendor-neutral format for 3D imaging data including point clouds. Designed to be hardware-agnostic, so data from any scanner can be stored without proprietary dependencies.

Note: some sensors use *proprietary* formats (e.g. Velodyne's PCAP) that require knowledge of the sensor's own geometry to convert to a standard format.

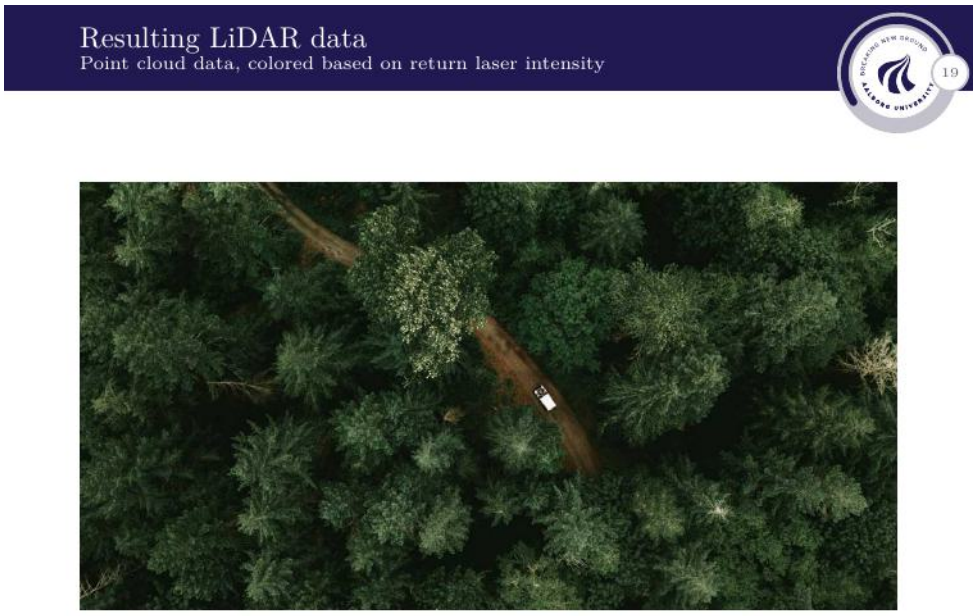


Figure 17: Aerial LiDAR point cloud coloured by return intensity. Bright points = highly reflective surfaces (road markings, vehicle roofs). Dark points = absorbing surfaces (vegetation, dark asphalt). The car on the forest track is clearly visible due to its high reflectivity relative to the surrounding canopy. Note: this is *not* real colour — it is a false-colour map of a single intensity number per point.

**The image looks coloured — but there is no real colour in LiDAR**

This is a common source of confusion. A standard LiDAR sensor such as the VLP-16 uses a **single laser wavelength** (903 nm near-infrared). It records *one number* per point — the return intensity $A_i$ (how much laser power came back). There is no red, green, or blue channel; no colour information whatsoever.

What you see in the image is a **false-colour map**: the software takes the intensity number and maps it to a colour scale (e.g. high intensity → bright/white, low intensity → dark/black). It is exactly like colouring a terrain map by elevation — the colours are chosen to make differences visible, they do not represent physical colours of the surfaces.

The differences in brightness come entirely from **how reflective each surface is at 903 nm near-infrared**:

- Road markings, metal surfaces, vehicle roofs → strong reflectors → bright
- Dark asphalt, vegetation → absorb near-IR → dark
- The car stands out brightly against the dark canopy for exactly this reason — not because it has a different colour, but because its surface reflects far more 903 nm laser power

The multi-wavelength LiDAR mentioned in Section 1.3 is a separate, more advanced technique where two wavelengths are fired simultaneously and the *ratio* of the two intensity values is used to classify materials. But that is not what is shown here — this image is purely single-wavelength intensity, false-coloured for visualisation.

## 4.2 Real Hardware: Velodyne VLP-16

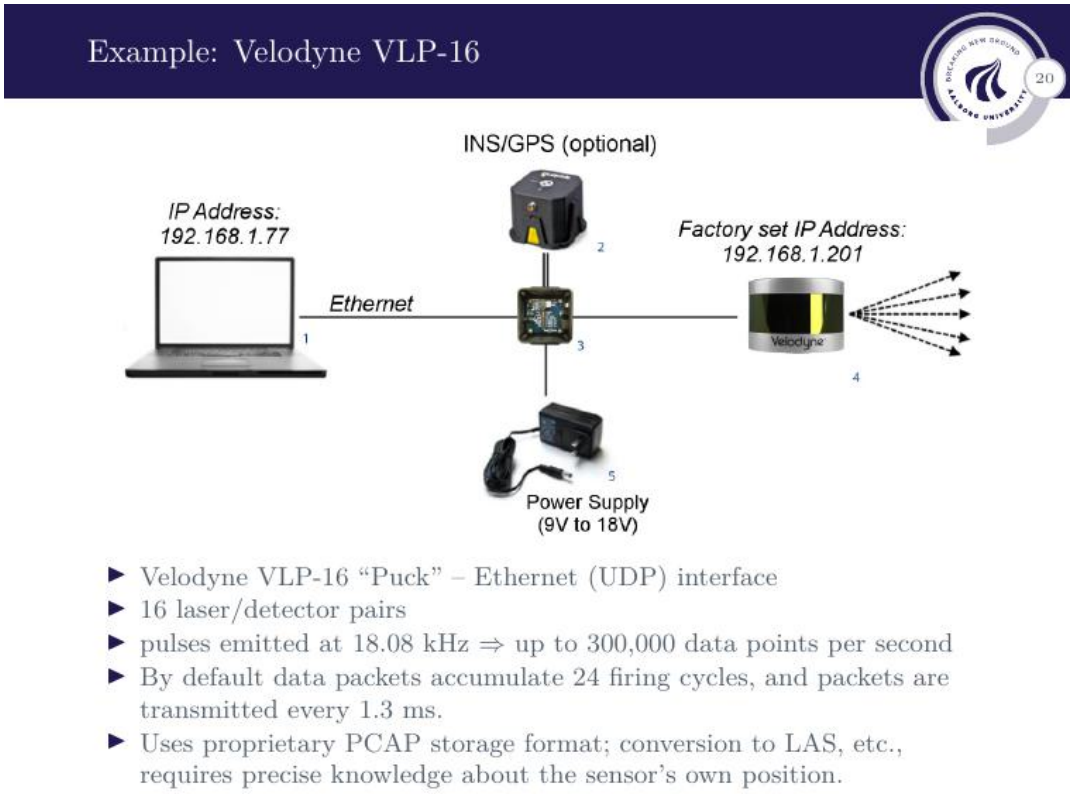


Figure 18: Velodyne VLP-16 "Puck" system setup. The sensor connects to a laptop via Ethernet (UDP packets). An optional INS/GPS unit provides georeferencing. The power supply runs from 9–18 V.

The **Velodyne VLP-16** (nicknamed the "Puck") is a compact rotating LiDAR sensor widely used in robotics and autonomous vehicles. Its key specifications:

- **16 laser/detector pairs** stacked at 16 different fixed **elevation angles**, ranging from $-15°$ to $+15°$ in $2°$ steps. As the head spins 360° horizontally, all 16 lasers fire at every azimuth step — not one per rotation, but all 16 *on every shot*. One full rotation therefore produces **16 concentric rings** of points, one ring per elevation angle, stacked vertically on top of each other. More lasers = more vertical rings = denser coverage of the scene height. The head completes roughly 10–20 full rotations per second.

- **Fire rate of 18.08 kHz** — each laser fires 18,080 times per second. With 16 lasers, this gives up to $16 \times 18{,}080 \approx$ **300,000 data points per second**.

- **Data packets every 1.3 ms** — by default, data accumulates over 24 firing cycles before being sent as a UDP packet over Ethernet. This means the host computer

receives a burst of points every 1.3 ms.

- **Interface: Ethernet (UDP)** — the sensor streams data directly over a standard network cable. No special hardware is needed on the host side.
- **Storage: PCAP format** — the raw UDP stream is typically captured in PCAP (packet capture) format. Converting to LAS or another standard format requires precise knowledge of the sensor's own mounting position, orientation, and timing, since the PCAP contains raw firing angles and distances — not fully-formed 3D coordinates.
- **Optional INS/GPS** — an Inertial Navigation System (INS) measures the sensor's acceleration and rotation continuously, tracking its position and orientation at all times. GPS adds absolute position in the world. Together they answer: *where was the sensor, and which way was it pointing, at the exact moment each pulse fired?*

  This is **optional for a static sensor** — if the sensor sits on a tripod at a known fixed location, its position and orientation are already known and no INS/GPS is needed.

  It becomes **essential for a moving platform** (robot, car, aircraft): without knowing the sensor's exact position at each firing moment, points recorded at different instants during motion get placed in wrong locations and the resulting map smears or distorts. The timestamp on each measurement packet is what links a LiDAR point to the correct INS/GPS position at that exact moment.

## 4.3 VLP-16 Data Packet Structure

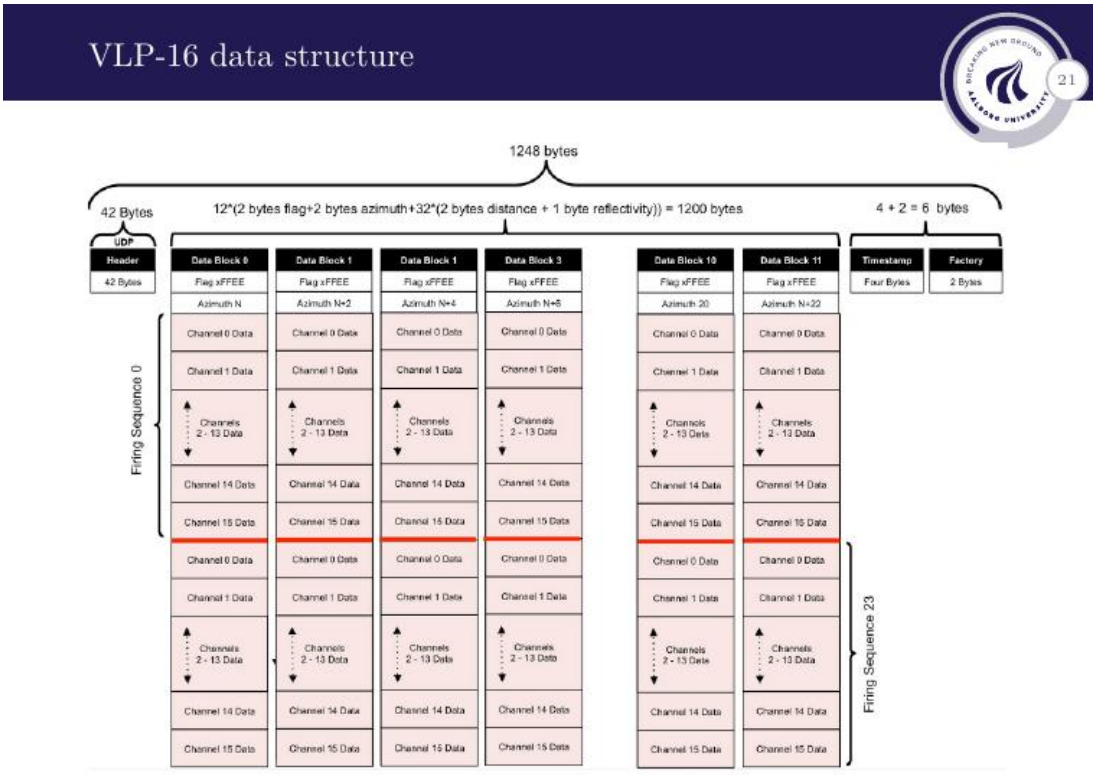


Figure 19: VLP-16 UDP data packet structure. Each packet is 1248 bytes and contains two firing sequences, each with 12 data blocks. Every data block stores one azimuth angle and 32 channel measurements (distance + reflectivity per channel).

Each UDP packet sent by the VLP-16 has a fixed size of **1248 bytes**, structured as follows:

- **42-byte header** — network and protocol overhead (IP/UDP headers).
- **1200 bytes of data** — $12 \times 2$ bytes (flag) + 2 bytes (azimuth) + $32 \times$ (2 bytes distance + 1 byte reflectivity) per data block, across 12 blocks per firing sequence and 2 firing sequences per packet. Each data block contains one azimuth reading and 32 channel measurements (distance and reflectivity for each of the 16 lasers, two returns per laser).
- **4-byte timestamp** — GPS-synchronised time of the packet.
- **2-byte factory field** — sensor model and return mode identifier.

**Why does the packet structure matter?**

When you capture raw PCAP data from the VLP-16, you receive a stream of these 1248-byte packets. To reconstruct a 3D point cloud you must:

1. Parse each packet to extract the azimuth angle and the 32 distance/reflectivity

values per block.

2. Know the *vertical angles* of each of the 16 laser beams (factory-calibrated, stored in the sensor's own memory).
3. Convert each (range, azimuth, elevation) polar measurement to Cartesian $(x, y, z)$.
4. Apply the timestamp and any platform motion correction (from INS/GPS) to place points in a global frame.

This is why conversion from PCAP to LAS requires "precise knowledge about the sensor's own position" as stated on the slide — the raw packet contains angles and distances, not world coordinates.

**Section 4 — Key Takeaways**

- Each point in a cloud carries: **3D position**, **timestamp**, **return intensity** (reflectivity), and optionally a **classification label**.
- Standard formats: **LAS** (industry standard), **LAZ** (compressed LAS), **E57** (vendor-neutral). Proprietary formats like PCAP require sensor-specific conversion.
- The Velodyne VLP-16 fires **16 lasers** at 18.08 kHz → up to 300,000 points/second, streamed as 1248-byte UDP packets every 1.3 ms.
- Raw packets contain **polar measurements** (range + azimuth). Converting to 3D world coordinates requires the sensor's factory-calibrated vertical beam angles and, for a moving platform, GPS/INS timestamps.

# Section 5 Robot Position Estimation with LiDAR

We now apply everything from the previous sections to a concrete engineering problem: a mobile robot equipped with a LiDAR must locate other robots in its environment. The robot sees a point cloud, must identify which clusters of points belong to which robot, fit a geometric model to each cluster, and then track the robots over time using a Kalman filter. This section follows that pipeline step by step.

## 5.1 Setup and Sensor Configuration

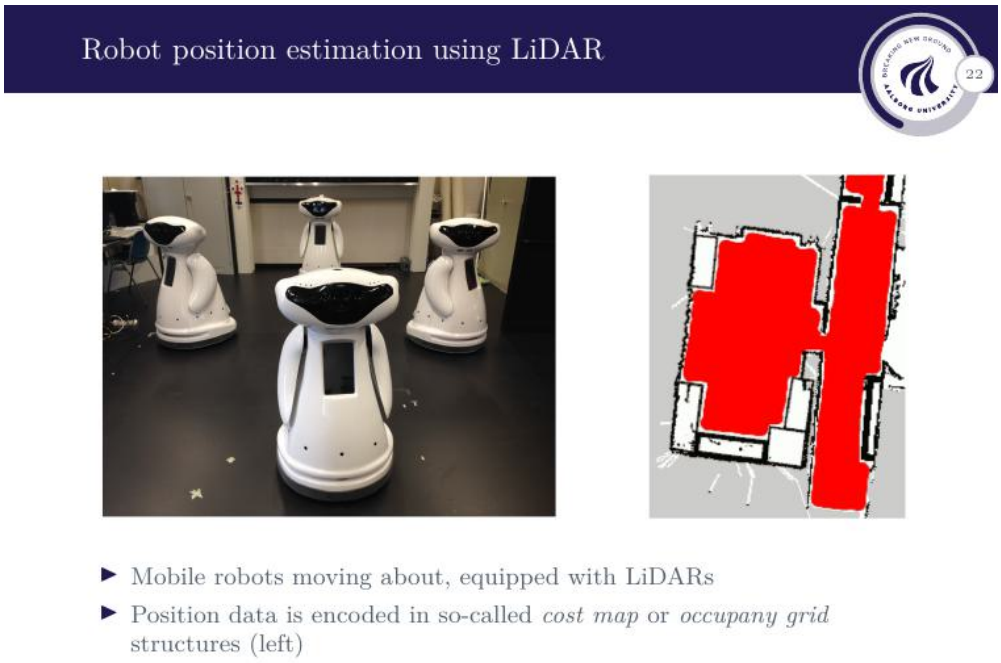


Figure 20: Mobile robots moving about in an environment, each equipped with a LiDAR. The map on the right is an *occupancy grid* (also called a cost map): each cell records whether that region of space is occupied, free, or unknown. The robot uses this to plan collision-free paths.

The robots carry a **Laser Range Finder (LRF)** — a 2D rotating LiDAR that sweeps a single horizontal plane. This is simpler than the full 3D VLP-16 from Section 4: rather than 16 stacked elevation angles, it fires at one fixed elevation and produces a single ring of range measurements as it rotates.

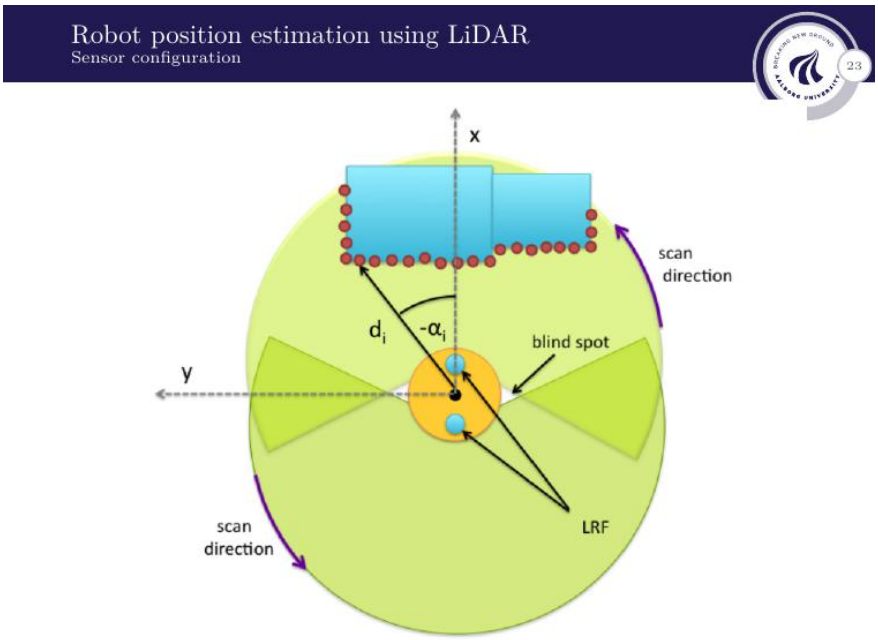


Figure 21: LRF sensor configuration viewed from above. The robot body is the orange circle at the centre. **Two LRF sensors** (blue dots) are mounted on opposite sides of the robot — one at the front and one at the rear. Each sensor sweeps its own arc (green wedges) in the scan direction, measuring range $d_i$ and angle $\alpha_i$ to each point it hits. The small white wedge is the **blind spot**: the narrow gap between the two sensor fields of view where neither sensor can see, typically blocked by the robot body itself.

A single 2D LRF typically only covers roughly 270° of arc, leaving a blind spot behind it. By mounting **two sensors on opposite sides** of the robot body, the combined coverage is nearly full 360°, with only two small blind spots where the robot body blocks each sensor.

At each scan sample $k$, the combined sensor pair produces a **point cloud** in polar coordinates:

$$S_k = \{(d_{i,k},\, \alpha_{i,k})\}, \quad i = 0, 1, \ldots, N_S$$

where $d_{i,k}$ is the measured range and $\alpha_{i,k}$ is the angle of the $i$-th beam at time step $k$, with measurements from both sensors merged into one set.

## 5.2 Noise Model for the Measurements

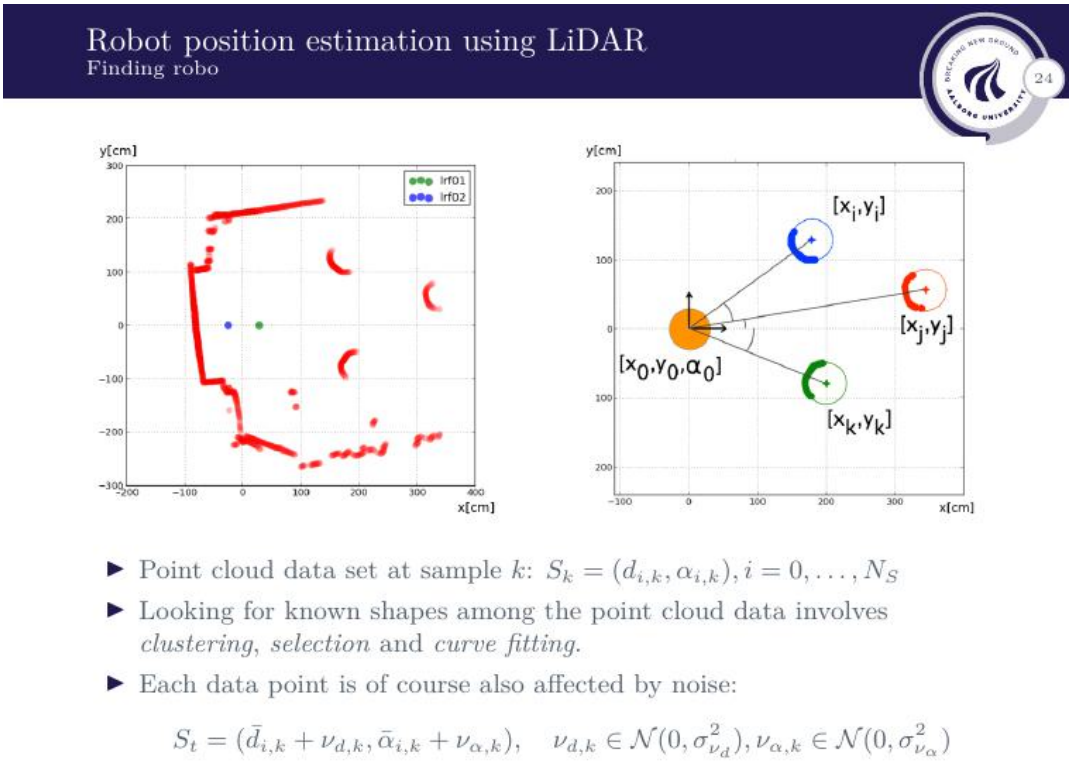


Figure 22: **Left:** full scan of the room. The **green dot** (lrf01) and **blue dot** (lrf02) mark the positions of the two LRF sensors mounted on the scanning robot — they appear close together because both sensors sit on the same robot body and the room is large. The **red points** are all the measurements from both sensors combined: the large continuous cloud is the room walls; the small crescent-shaped clusters scattered around are the *other* robots being detected. **Right:** zoomed view of the detection task. The scanning robot is at known position and heading $[x_0, y_0, \alpha_0]$. Three target robots have been detected at positions $[x_i, y_i]$, $[x_j, y_j]$, $[x_k, y_k]$ by fitting discs to the crescent clusters. The arrows show the line of sight from the scanning robot to each detected robot.

Every measurement is affected by noise. The *true* polar coordinates of a point are $(d_{i,k}, \alpha_{i,k})$; the *measured* (noisy) values are:

$$\tilde{S}_k = \left(\bar{d}_{i,k} + \nu_{d,k},\ \bar{\alpha}_{i,k} + \nu_{\alpha,k}\right)$$

where the noise terms are independent zero-mean Gaussians:

$$\nu_{d,k} \sim \mathcal{N}(0,\ \sigma^2_{\nu_d}) \qquad \nu_{\alpha,k} \sim \mathcal{N}(0,\ \sigma^2_{\nu_\alpha})$$

**What the noise model tells you.** The range measurement $\bar{d}_{i,k}$ has Gaussian noise with standard deviation $\sigma_{\nu_d}$ — a typical LiDAR might have $\sigma_{\nu_d} \approx 2$–$5\,\text{cm}$. The angle measurement $\bar{\alpha}_{i,k}$ has independent Gaussian noise with standard deviation $\sigma_{\nu_\alpha}$ — typically a fraction of a degree. The two noises are **uncorrelated** (independent): an error

in the range measurement tells you nothing about the angle error. This independence is captured by the diagonal noise covariance matrix introduced in Section 5.3.

## 5.3 Uncertainty Ellipsoid: Polar to Cartesian

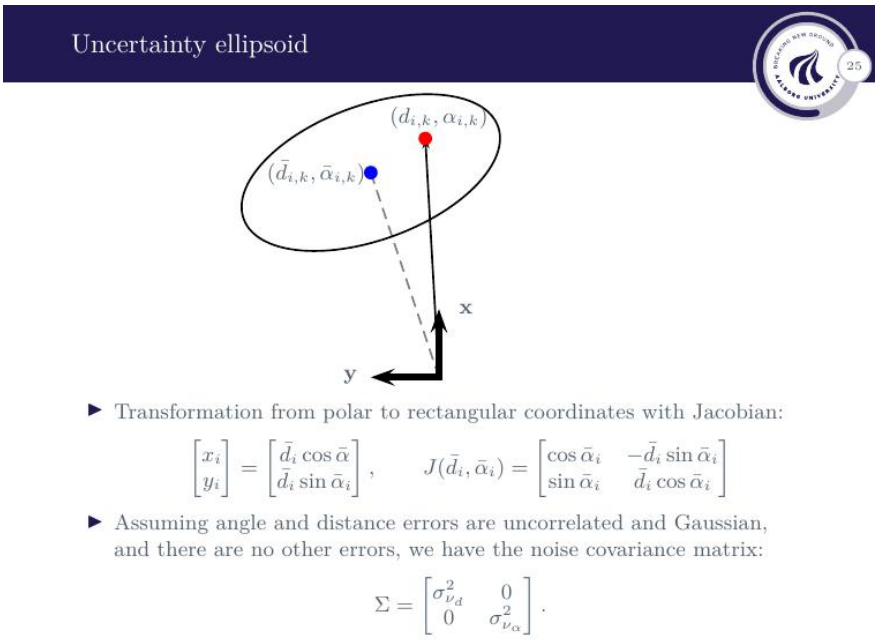


Figure 23: Uncertainty ellipsoid for a single LiDAR point. The red dot is the true point $\boldsymbol{\xi}_i$; the blue dot is the noisy measurement $\bar{\boldsymbol{\xi}}_i$. In polar coordinates the uncertainty region is a rectangle; after the nonlinear polar-to-Cartesian transformation it becomes a tilted ellipse in $(x, y)$ space.

### Step 1 — Why do we need to convert to Cartesian at all?

The LRF natively produces polar measurements $(d_i, \alpha_i)$. Everything downstream — clustering, disc fitting, and the Kalman filter — works in **Cartesian** $(x, y)$ coordinates. So we must convert each polar point to Cartesian. The conversion is:

$$\boldsymbol{\xi}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix} = \begin{bmatrix} \bar{d}_i \cos\bar{\alpha}_i \\ \bar{d}_i \sin\bar{\alpha}_i \end{bmatrix}$$

This is straightforward for the *position*. The problem arises when we also want to convert the *uncertainty*.

### Step 2 — Why can we not just plug the noise directly into the conversion formula?

The conversion involves sin and cos — it is **nonlinear**. With a nonlinear function you cannot simply say "range noise $\sigma_{\nu_d}$ maps to Cartesian noise $\sigma_{\nu_d}$" because the mapping from polar error to Cartesian error changes depending on the current angle $\alpha_i$. A range error at $\alpha_i = 0$ moves the point purely in $x$; the same range error at $\alpha_i = 90°$ moves it purely in $y$. The direction and magnitude of the Cartesian uncertainty depend on *where the beam is pointing* — and you cannot capture that with a simple scalar multiplication.

**Step 3 — The noise covariance matrix $\Sigma$ (defined in polar).**

First, we describe the uncertainty *in the polar domain*. Since range and angle errors are independent Gaussians (from Section 5.2), the **polar noise covariance matrix** is diagonal:

$$\Sigma = \begin{bmatrix} \sigma_{\nu_d}^2 & 0 \\ 0 & \sigma_{\nu_\alpha}^2 \end{bmatrix}$$

This matrix is **in polar coordinates** — it is a $2 \times 2$ matrix whose axes are range $d$ and angle $\alpha$, not $x$ and $y$. The off-diagonal zeros reflect that range and angle errors are completely independent: knowing one error tells you nothing about the other. In polar space, the uncertainty region at probability level $p$ is therefore a simple **axis-aligned rectangle**: $\pm\sigma_{\nu_d}$ in range, $\pm\sigma_{\nu_\alpha}$ in angle.

**Step 4 — The Jacobian linearises the conversion.**

To transform $\Sigma$ from polar into Cartesian we linearise the conversion at the current measured point $(\bar{d}_i, \bar{\alpha}_i)$ — exactly the same idea as the EKF Jacobian from Lecture 4. The Jacobian $J$ is the matrix of partial derivatives of $(x, y)$ with respect to $(d, \alpha)$:

$$J(\bar{d}_i, \bar{\alpha}_i) = \frac{\partial\,[x,\, y]}{\partial\,[d,\, \alpha]} = \begin{bmatrix} \cos\bar{\alpha}_i & -\bar{d}_i \sin\bar{\alpha}_i \\ \sin\bar{\alpha}_i & \bar{d}_i \cos\bar{\alpha}_i \end{bmatrix}$$

Each column tells you how a small nudge in one polar variable moves the Cartesian point:

- **First column** $(\partial/\partial d)$: a small increase in range moves the point in the direction $[\cos\alpha_i, \sin\alpha_i]$ — directly along the beam.
- **Second column** $(\partial/\partial\alpha)$: a small increase in angle rotates the point perpendicular to the beam by $\bar{d}_i \cdot [-\sin\alpha_i, \cos\alpha_i]$ — the $\bar{d}_i$ factor means this grows with range.

The Cartesian covariance is then approximated as:

$$\mathrm{Cov}_{\mathrm{Cartesian}} \approx J\,\Sigma\,J^\top$$

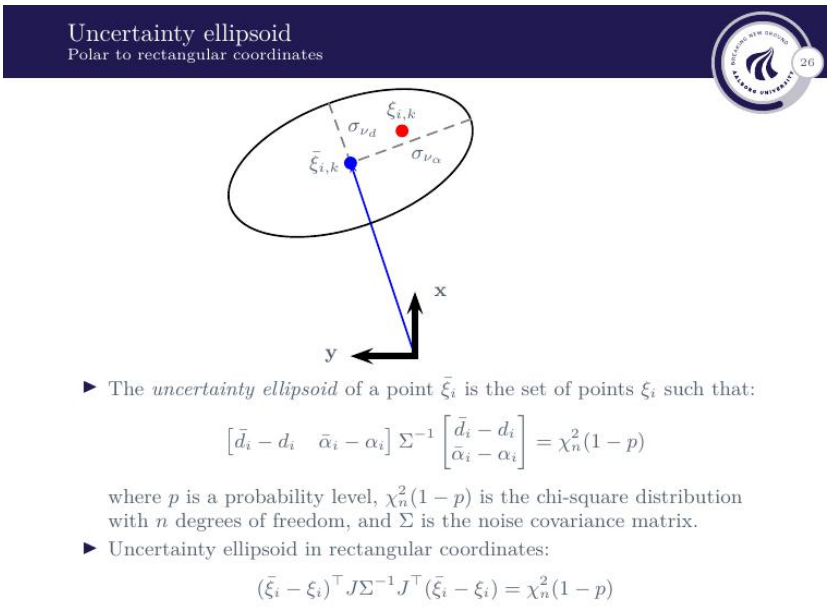


Figure 24: Slide 26: the uncertainty ellipse in Cartesian coordinates. The blue dot $\bar{\boldsymbol{\xi}}_{i,k}$ is the noisy Cartesian measurement; the red dot $\boldsymbol{\xi}_{i,k}$ is the true position. The ellipse is the set of all Cartesian points that are consistent with the measurement at probability level $p$. $\sigma_{\nu_d}$ controls the *radial* (along-beam) axis; $\sigma_{\nu_\alpha}$ controls the *tangential* (across-beam) axis, which grows with range because of the $\bar{d}_i$ factor in $J$.

### Step 5 — Uncertainty ellipsoid in polar coordinates.

We now ask: *given a noisy measurement* $(\bar{d}_i, \bar{\alpha}_i)$*, what is the region of true values* $(d_i, \alpha_i)$ *that could plausibly have produced it?* The answer comes from the noise model in Section 5.2: the errors $\nu_d = \bar{d}_i - d_i$ and $\nu_\alpha = \bar{\alpha}_i - \alpha_i$ are independent Gaussians. From statistics: the sum of $n$ independent *squared standardised* Gaussians follows a **chi-squared distribution** with $n$ degrees of freedom. Setting that sum equal to a threshold $\chi^2_n(1-p)$ draws a boundary that encloses $(1-p)$ of the probability mass — the **confidence ellipsoid**:

$$\begin{bmatrix}\bar{d}_i - d_i & \bar{\alpha}_i - \alpha_i\end{bmatrix} \Sigma^{-1} \begin{bmatrix}\bar{d}_i - d_i \\ \bar{\alpha}_i - \alpha_i\end{bmatrix} = \chi^2_n(1-p)$$

*Expanding the matrix form.* Because $\Sigma$ is diagonal (Step 3), so is its inverse:

$$\Sigma^{-1} = \begin{bmatrix}1/\sigma^2_{\nu_d} & 0 \\ 0 & 1/\sigma^2_{\nu_\alpha}\end{bmatrix}$$

Multiplying out the quadratic form gives a plain scalar equation:

$$\frac{(\bar{d}_i - d_i)^2}{\sigma^2_{\nu_d}} + \frac{(\bar{\alpha}_i - \alpha_i)^2}{\sigma^2_{\nu_\alpha}} = \chi^2_n(1-p)$$

This is exactly the standard ellipse equation $\dfrac{X^2}{a^2} + \dfrac{Y^2}{b^2} = 1$, with semi-axis lengths:

$$a = \sigma_{\nu_d}\sqrt{\chi^2_n(1-p)}, \qquad b = \sigma_{\nu_\alpha}\sqrt{\chi^2_n(1-p)}$$

Because there is *no cross-term* $(\bar{d}_i - d_i)(\bar{\alpha}_i - \alpha_i)$ the two axes point exactly along $d$ and $\alpha$: the ellipse is **axis-aligned** — zero tilt. This makes sense: range and angle errors are independent (Step 3), so they do not "push" each other sideways.

**Understanding the chi-squared threshold $\chi^2_n(1-p)$**

- $n = 2$: we have two polar variables ($d$ and $\alpha$), so $n = 2$ degrees of freedom.
- $p$ is the probability we are willing to *exclude*: choosing $p = 0.05$ means we want a **95 % confidence region**.
- Numerical example: $\chi^2_2(0.95) \approx 5.99$. The semi-axis lengths become $\sigma_{\nu_d}\sqrt{5.99} \approx 2.45\,\sigma_{\nu_d}$ and $\sigma_{\nu_\alpha}\sqrt{5.99} \approx 2.45\,\sigma_{\nu_\alpha}$.
- Think of it as a *scaling knob*: a larger $\chi^2$ gives a bigger ellipse (more probability captured); a smaller $\chi^2$ gives a smaller ellipse (less probability but tighter bound).
- **Physical reading**: the ellipse is centred on the noisy measurement $(\bar{d}_i, \bar{\alpha}_i)$. The true point $(d_i, \alpha_i)$ lies inside this ellipse with 95 % probability.

**Step 6 — Uncertainty ellipsoid in Cartesian coordinates.**

Step 5 gives a clean axis-aligned ellipse *in polar space*. Now we want the equivalent region in Cartesian $(x, y)$, because that is what clustering, disc fitting, and the Kalman filter all work with.

*How do we get there?* From Step 4, the Jacobian $J$ tells us how a small polar error $\Delta m_i = [\bar{d}_i - d_i,\ \bar{\alpha}_i - \alpha_i]^\top$ maps to a Cartesian error:

$$\underbrace{(\bar{\boldsymbol{\xi}}_i - \boldsymbol{\xi}_i)}_{\text{Cartesian error}} \approx J\ \underbrace{[\bar{d}_i - d_i,\ \bar{\alpha}_i - \alpha_i]^\top}_{\text{polar error}}$$

Substituting the polar error (expressed in terms of the Cartesian error) back into Step 5's quadratic form and rearranging replaces $\Sigma^{-1}$ with the **Cartesian precision matrix** $J\Sigma^{-1}J^\top$:

$$(\bar{\boldsymbol{\xi}}_i - \boldsymbol{\xi}_i)^\top\ J\Sigma^{-1}J^\top\ (\bar{\boldsymbol{\xi}}_i - \boldsymbol{\xi}_i) = \chi^2_n(1-p)$$

The $\chi^2_n(1-p)$ threshold is **unchanged** — changing coordinate systems does not change probability levels.

*Why is this ellipse now tilted instead of axis-aligned?* In Step 5, $\Sigma^{-1}$ was diagonal: its axes pointed along $d$ and $\alpha$ with zero tilt. After multiplying by $J$ and $J^\top$, the matrix $J\Sigma^{-1}J^\top$ is **no longer diagonal**: it contains $\sin\bar{\alpha}_i$ and $\cos\bar{\alpha}_i$ (evaluated at the actual beam angle), which *mixes* the range and angle uncertainties. The natural axes of this new matrix are:

- **Radial axis** — along the beam, direction $[\cos\bar{\alpha}_i, \sin\bar{\alpha}_i]$. This axis captures the *range* error $\sigma_{\nu_d}$. Range noise is already measured in *metres*: if the sensor reads the distance 3 cm too long, the point moves 3 cm directly along the beam — regardless of whether the object is 2 m or 20 m away. So the half-length of this axis is simply $\approx \sigma_{\nu_d}\sqrt{\chi^2}$ — **constant, does not grow with range**.

- **Tangential axis** — perpendicular to the beam, direction $[-\sin\bar{\alpha}_i, \cos\bar{\alpha}_i]$. This axis captures the *angle* error $\sigma_{\nu_\alpha}$. Angle noise is measured in *radians* — but what we care about is the sideways position error in *metres*. Think of the object sitting at the tip of the laser beam: if the beam swings by a small angle $\sigma_{\nu_\alpha}$, the tip traces an arc of

length:

$$\text{arc} = \bar{d}_i \times \sigma_{\nu_\alpha}$$

(arc length = radius × angle). At 2 m range a 0.01 rad error moves the point 0.02 m sideways; at 20 m the same 0.01 rad moves it 0.20 m sideways. So the half-length is $\approx \bar{d}_i \, \sigma_{\nu_\alpha} \sqrt{\chi^2}$ — **grows linearly with range**.

Unless the beam points exactly along the $x$- or $y$-axis, neither of these directions is horizontal or vertical — the ellipse is therefore **tilted** by exactly the beam angle $\bar{\alpha}_i$.

**Comparing the two ellipsoids side by side**

| | **Polar (Step 5)** | **Cartesian (Step 6)** |
|---|---|---|
| Error vector | $[\bar{d}_i - d_i,\ \bar{\alpha}_i - \alpha_i]^\top$ | $\bar{\boldsymbol{\xi}}_i - \boldsymbol{\xi}_i$ |
| Weight matrix | $\Sigma^{-1}$ (diagonal) | $J\Sigma^{-1}J^\top$ (full, off-diagonal) |
| Ellipse tilt | Axis-aligned along $d$, $\alpha$ | Tilted by beam angle $\bar{\alpha}_i$ |
| Radial half-axis | $\sigma_{\nu_d}\sqrt{\chi^2}$ | $\sigma_{\nu_d}\sqrt{\chi^2}$ (same) |
| Tangential half-axis | $\sigma_{\nu_\alpha}\sqrt{\chi^2}$ | $\bar{d}_i \, \sigma_{\nu_\alpha}\sqrt{\chi^2}$ *(grows!)* |

**Why does the tangential axis grow?** An angular error of 1 radian at range $\bar{d}_i$ sweeps an arc of length $\bar{d}_i \times 1\,\text{rad} = \bar{d}_i$ metres. So the same angle noise produces a *larger sideways position error* the further away the object is. At short range the ellipse is compact in all directions (you know both distance and lateral position well). At long range it becomes very elongated sideways: you know the distance to the object accurately, but its lateral position is increasingly uncertain — the tangential axis keeps growing while the radial axis stays fixed.

## 5.4 Recognising Objects: Clustering and Disc Fitting

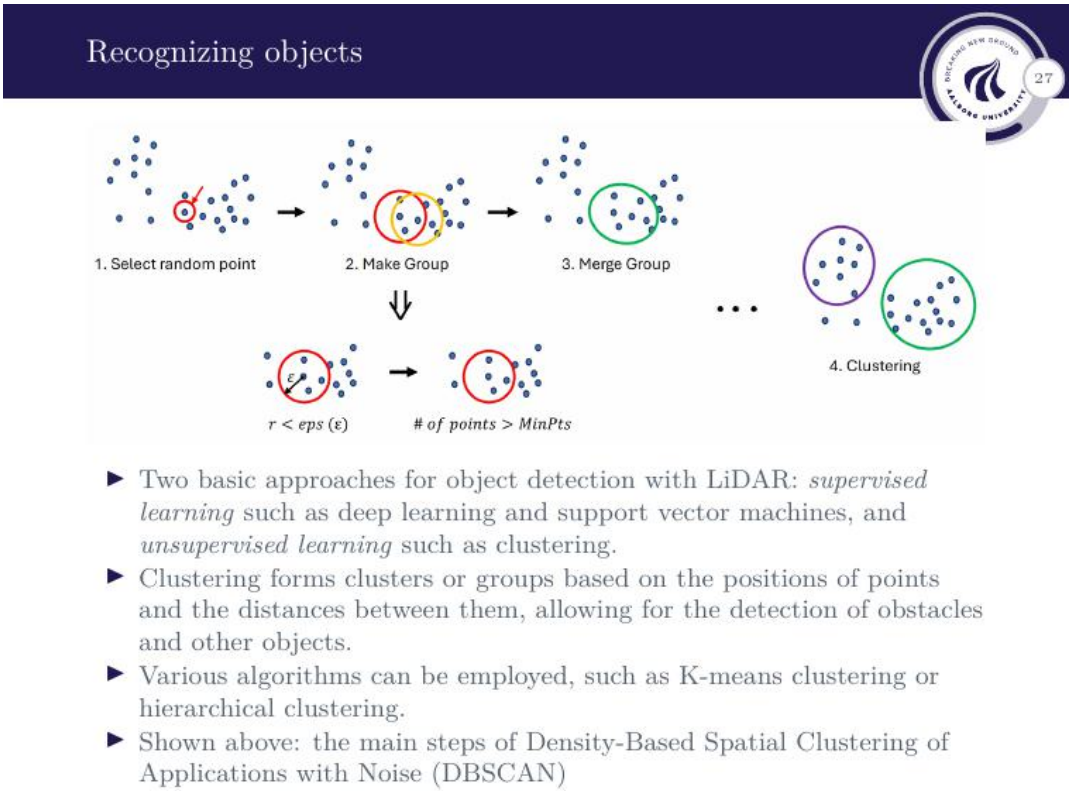


Figure 25: DBSCAN clustering steps: (1) select a random point, (2) group nearby points within radius $\varepsilon$, (3) merge overlapping groups, repeat until all points are assigned. The result (4) is a set of labelled clusters, one per detected object.

Given the raw point cloud $S_k$, the first task is to find which points belong to which robot. This involves three steps:

**Step 1 — Clustering with DBSCAN.**

**DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) groups points by how densely packed they are. It has two parameters:

- $\varepsilon$ — the *neighbourhood radius*: how close two points must be to be considered neighbours.
- MinPts — the *minimum neighbour count*: how many neighbours a point needs to be able to "grow" a cluster.

*How it works, step by step:*

1. Pick any unvisited point $P$.
2. Count how many other points lie within radius $\varepsilon$ of $P$ — call this the *neighbourhood* of $P$.
3. If the neighbourhood contains *fewer* than MinPts points: mark $P$ as **noise** for now (it may still be rescued later by another cluster passing through it).

4. If the neighbourhood contains *at least* MinPts points: $P$ is a **core point** — start a new cluster and add $P$ plus all its neighbours to it.

5. For every new point $Q$ just added to the cluster: look at $Q$'s own neighbourhood. If $Q$ is also a core point ($\geq$ MinPts neighbours), pull *its* neighbours into the cluster too. Keep doing this until no new points can be added — the cluster has been fully expanded.

6. Move to the next unvisited point and repeat from step 1.

After all points are visited, every point is either:

- **Core point**: $\geq$ MinPts neighbours, can expand the cluster.
- **Border point**: $<$ MinPts neighbours, but within $\varepsilon$ of a core point — part of the cluster, just can't grow it further.
- **Noise**: not within $\varepsilon$ of any core point — excluded entirely.

**Why DBSCAN works well for LiDAR point clouds**

Each robot in the scan reflects several LiDAR points on its surface (an arc of points), all close together. Between two robots there is empty space — no LiDAR returns. DBSCAN exploits exactly this:

- Points on the *same robot* are close (within $\varepsilon$) $\rightarrow$ they join the same cluster.
- Points on *different robots* are separated by a gap larger than $\varepsilon$ $\rightarrow$ they form separate clusters.
- A single stray reflection has no neighbours $\rightarrow$ labelled noise and ignored.

The result is one cluster per robot — no need to know in advance how many robots there are (unlike k-means, which requires you to specify the number of groups).

## Disc Fitting

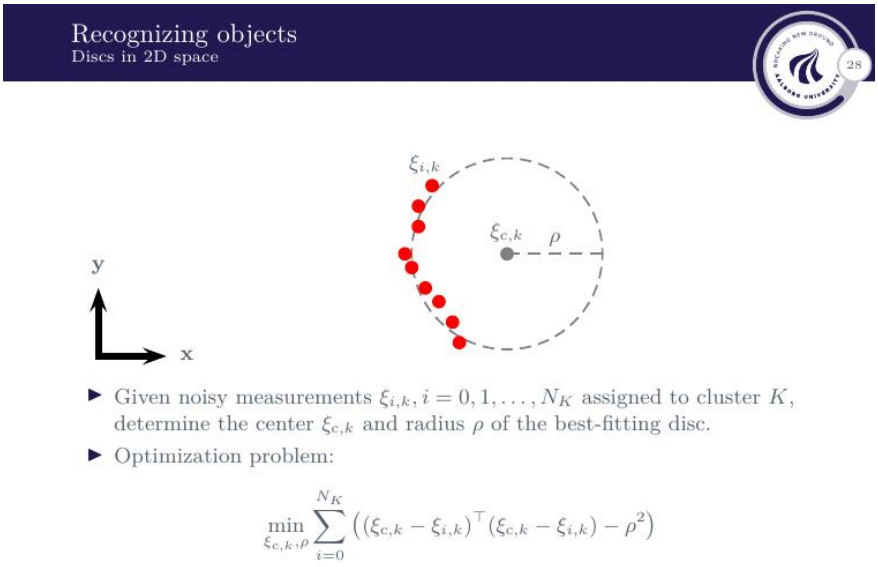


Figure 26: Disc model for a robot cluster. Given $N_K$ noisy 2D points $\xi_{i,k}$ belonging to cluster $K$, find the centre $\xi_{c,k} = (x_{c,k}, y_{c,k})$ and radius $\rho$ of the best-fitting disc.

Each robot is modelled as a **disc** (circle in 2D). Given $N_K$ noisy Cartesian points $\boldsymbol{\xi}_{i,k}$ assigned to cluster $K$, the best-fitting disc centre $\boldsymbol{\xi}_{c,k}$ and radius $\rho$ are found by minimising:

$$\min_{\boldsymbol{\xi}_{c,k},\rho} \sum_{i=0}^{N_K} \left[(\boldsymbol{\xi}_{c,k} - \boldsymbol{\xi}_{i,k})^\top (\boldsymbol{\xi}_{c,k} - \boldsymbol{\xi}_{i,k}) - \rho^2\right]^2$$

This is a least-squares problem: minimise the sum of squared differences between the squared distances from each point to the candidate centre and the squared radius.

> **What are we trying to minimise, and what do we get?**
>
> We are trying to find the **disc centre** $(x_c, y_c)$ **and radius** $\rho$ that best explain all the noisy points in the cluster. The cost function penalises every point that does *not* sit exactly on the circle: if a point is too far from the centre (further than $\rho$) or too close, it adds to the error. Minimising the total error over all $N_K$ points gives the **best-fit circle** through the cluster.
>
> What we get out: the disc centre $(x_{c,k}, y_{c,k})$, which is the **estimated robot position** at time step $k$. This is then passed to the Kalman filter for tracking.

**Reformulating as a linear problem.**

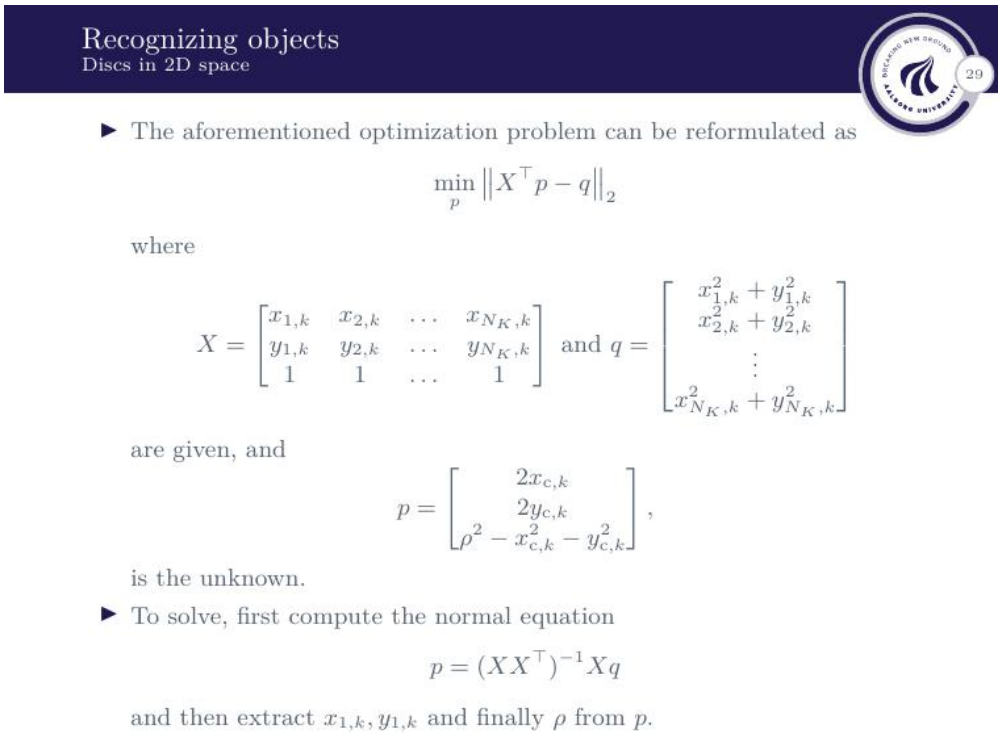


Figure 27: Reformulation of the disc-fitting problem as a linear least-squares system $\min_p \|X^\top p - q\|_2$. The matrix $X$ and vector $q$ are built from the measured point coordinates; $p$ contains the unknowns (centre and radius).

*Why is the original problem nonlinear?* A perfect circle satisfies $(x_i - x_c)^2 + (y_i - y_c)^2 = \rho^2$ for every point $i$. The unknowns $x_c$, $y_c$ sit *inside* squared terms, so minimising the error directly requires iterative nonlinear solvers. The trick is to expand the bracket algebraically and rearrange so that the unknowns appear only *multiplied by known numbers* (i.e. linearly).

*The algebraic trick — expand and rearrange.* Start from the ideal circle equation for point $i$:

$$(x_{i,k} - x_{c,k})^2 + (y_{i,k} - y_{c,k})^2 = \rho^2$$

Expand:

$$x_{i,k}^2 - 2x_{i,k}x_{c,k} + x_{c,k}^2 + y_{i,k}^2 - 2y_{i,k}y_{c,k} + y_{c,k}^2 = \rho^2$$

Move all the *known* data terms $(x_{i,k}^2 + y_{i,k}^2)$ to the right and all *unknown* terms to the left:

$$2x_{c,k} \cdot x_{i,k} + 2y_{c,k} \cdot y_{i,k} + \underbrace{(\rho^2 - x_{c,k}^2 - y_{c,k}^2)}_{\text{one combined unknown}} \cdot 1 = x_{i,k}^2 + y_{i,k}^2$$

Now define three new unknowns that group the messy parts together:

$$p_1 = 2x_{c,k}, \qquad p_2 = 2y_{c,k}, \qquad p_3 = \rho^2 - x_{c,k}^2 - y_{c,k}^2$$

The equation for point $i$ becomes:

$$p_1 \cdot x_{i,k} \; + \; p_2 \cdot y_{i,k} \; + \; p_3 \cdot 1 \; = \; x_{i,k}^2 + y_{i,k}^2$$

This is **linear** in $(p_1, p_2, p_3)$ — all the unknowns appear only multiplied by known numbers.

*Writing all $N_K$ equations as one matrix system.* We have one equation per point. Stack them all into a matrix form $X^\top p = q$:

$$\underbrace{\begin{bmatrix} x_{1,k} & y_{1,k} & 1 \\ x_{2,k} & y_{2,k} & 1 \\ \vdots & \vdots & \vdots \\ x_{N_K,k} & y_{N_K,k} & 1 \end{bmatrix}}_{X^\top\ (N_K \times 3)} \underbrace{\begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix}}_{p\ (3\times 1)} = \underbrace{\begin{bmatrix} x_{1,k}^2 + y_{1,k}^2 \\ x_{2,k}^2 + y_{2,k}^2 \\ \vdots \\ x_{N_K,k}^2 + y_{N_K,k}^2 \end{bmatrix}}_{q\ (N_K \times 1)}$$

**What each matrix and vector actually contains**

$X^\top$ $(N_K \times 3)$ — **one row per point:**

- Column 1: the $x$-coordinate of each Cartesian point (will be multiplied by $p_1 = 2x_c$).
- Column 2: the $y$-coordinate of each Cartesian point (will be multiplied by $p_2 = 2y_c$).
- Column 3: all ones (will be multiplied by $p_3$, the combined constant term).

Equivalently, $X$ is $3 \times N_K$ with each *column* being one point $[x_{i,k},\ y_{i,k},\ 1]^\top$.

$q$ $(N_K \times 1)$ — **one entry per point:** Each entry is $x_{i,k}^2 + y_{i,k}^2$ — the *squared distance from the origin* to point $i$. This is the right-hand side that came from moving the data terms across in the rearrangement above. It is built entirely from known measured values.

$p$ $(3 \times 1)$ — **the three unknowns:**

- $p_1 = 2x_{c,k}$ $\quad \Rightarrow$ extract centre $x_{c,k} = p_1/2$
- $p_2 = 2y_{c,k}$ $\quad \Rightarrow$ extract centre $y_{c,k} = p_2/2$
- $p_3 = \rho^2 - x_{c,k}^2 - y_{c,k}^2$ $\quad \Rightarrow$ extract radius $\rho = \sqrt{p_3 + x_{c,k}^2 + y_{c,k}^2}$

*Why is the system overdetermined, and what does the normal equation do?*

We have $N_K$ equations (one per point) but only 3 unknowns. As long as $N_K > 3$ — which is always true for a useful cluster — there is no vector $p$ that satisfies all equations exactly, because the measured points are noisy. Instead we **minimise the total squared residual** $\|X^\top p - q\|^2$: find the $p$ whose predicted values $X^\top p$ are as close as possible to the measured right-hand side $q$.

Differentiating $\|X^\top p - q\|^2$ with respect to $p$ and setting it to zero gives the **normal equation**:

$$XX^\top p = Xq \quad \Longrightarrow \quad p = (XX^\top)^{-1} Xq$$

Note that $XX^\top$ is only $3 \times 3$ (from a $3 \times N_K$ times $N_K \times 3$ product), regardless of how many points there are — so the matrix inversion is tiny. This gives the **exact global minimum in one step**: no iteration, no initial guess needed.

**What do you actually get out of this minimisation?**

You get the disc centre $(x_{c,k},\, y_{c,k})$ — the **best estimate of the robot's position** given all $N_K$ noisy measurements in that cluster.

Think of it this way:

- Each individual LiDAR point on the robot's surface is *noisy* — it has its own uncertainty ellipsoid from Section 5.3. You could use one point as your position guess, but it would be unreliable.
- The disc fitting uses *all* $N_K$ points at once. The noise on different points is independent — some measure a bit too far, some a bit too close, some shifted left, some shifted right. By fitting a circle through all of them together, those random errors largely *cancel out*, and the centre estimate becomes much more reliable than any single point alone.
- The more points in the cluster, the better: noise averages out faster with more measurements.

So the result is not truly "noise-free" — it is the **maximum likelihood estimate** of the disc centre: the most probable true robot position consistent with all the noisy data. This estimated centre $(x_{c,k}, y_{c,k})$ is then handed to the **Kalman filter** as the position measurement for that time step.

## 5.5 Simulation and Tracking Moving Targets

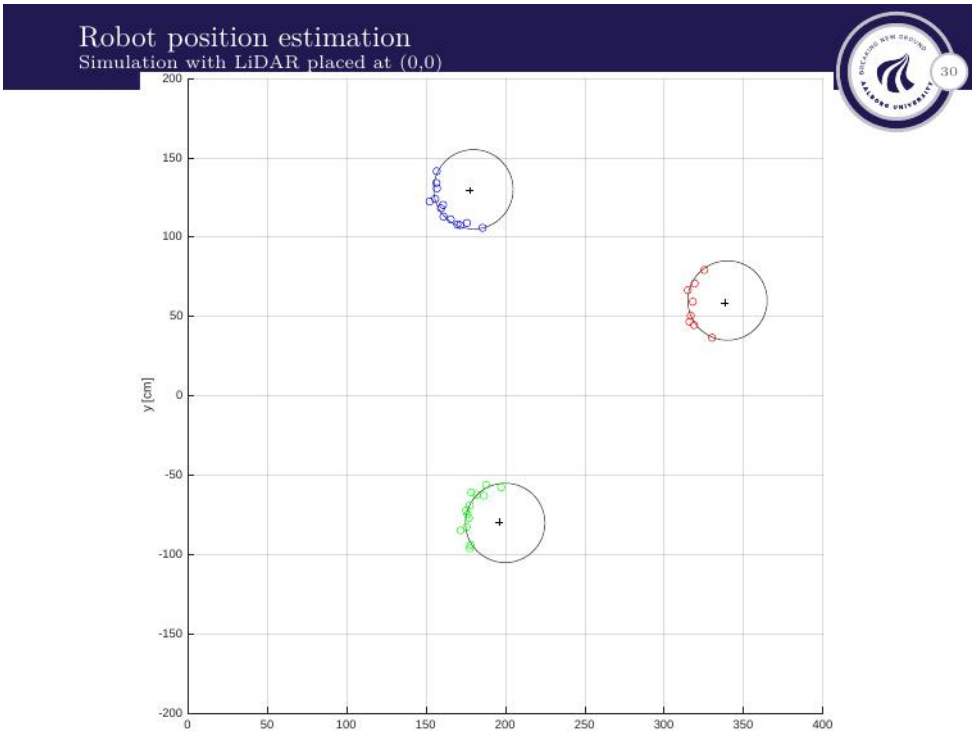


Figure 28: Simulation result with the LiDAR placed at the origin $(0, 0)$. Three robots (blue, green, red) are detected from partial arc-shaped point clusters on their surfaces. The fitted disc centres (+ markers) closely match the true robot positions. Note that each robot only reflects a partial arc — the LiDAR only sees the side facing the sensor.

Once the disc centre $\boldsymbol{\xi}_{c,k} = (x_{c,k}, y_{c,k})$ of each robot is found at each time step $k$, the position estimate can be fed into a **Kalman filter** to track the robots as they move.

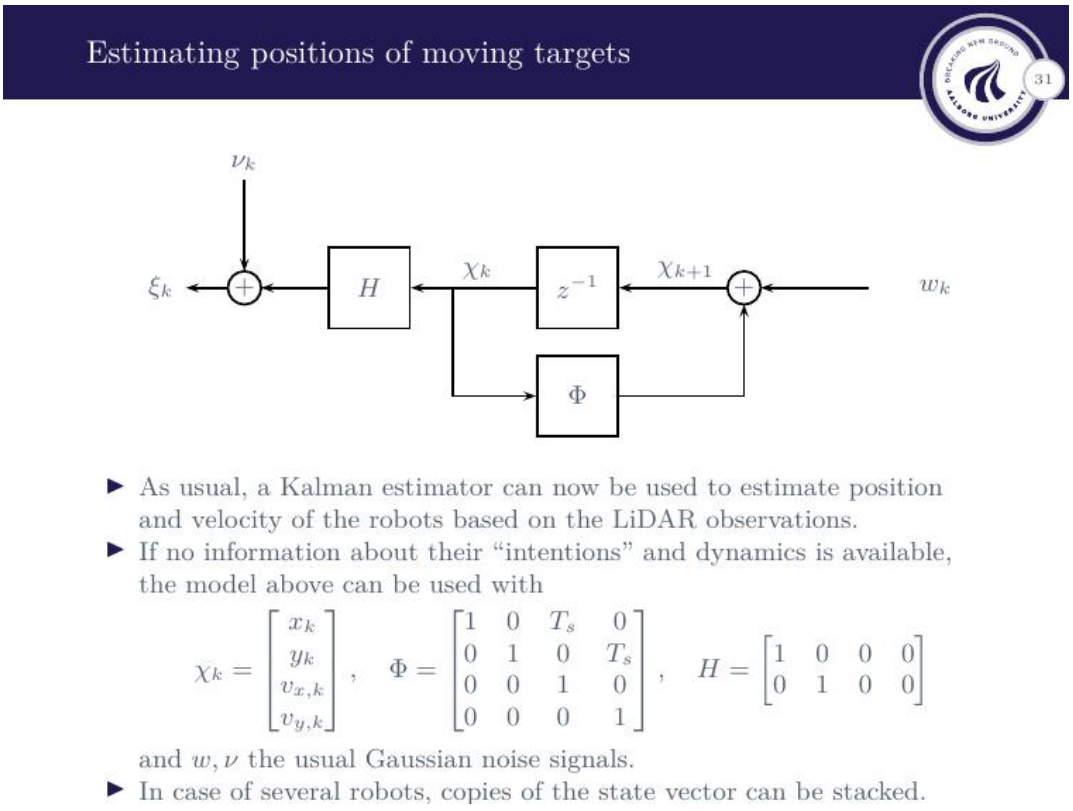


Figure 29: State-space model for a moving robot. The state vector contains position and velocity; $\Phi$ propagates the state forward by one time step $T_s$; $H$ extracts the position measurements from the state.

**State-space model.** If no information about the robots' dynamics is available, a simple constant-velocity model is used. The state vector for one robot is:

$$\chi_k = \begin{bmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \end{bmatrix}$$

containing position $(x_k, y_k)$ and velocity $(v_{x,k}, v_{y,k})$.

The state transition matrix (assuming constant velocity over one step $T_s$):

$$\Phi = \begin{bmatrix} 1 & 0 & T_s & 0 \\ 0 & 1 & 0 & T_s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Each position updates by adding velocity $\times$ time step; velocity is assumed constant (changed only by process noise $w_k$).

The measurement matrix extracts only the position from the state, since the LiDAR measures position (disc centre), not velocity:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

The full discrete-time model is:

$$\chi_{k+1} = \Phi\,\chi_k + w_k, \qquad \xi_k = H\,\chi_k + \nu_k$$

where $w_k$ and $\nu_k$ are Gaussian noise signals. A standard Kalman filter can now be applied to estimate position and velocity from the noisy disc-centre measurements.

**Why use a Kalman filter here?**

The disc-fitting step gives a noisy position estimate $\boldsymbol{\xi}_{c,k}$ at each time step. On its own, each estimate has uncertainty from the LiDAR noise (via the uncertainty ellipsoid from Section 5.3). The Kalman filter does two things:

- **Smooths** the position estimate by combining the current measurement with the prediction from the dynamic model — a single noisy measurement is less trusted than the combination of many measurements over time.
- **Estimates velocity** even though velocity is never directly measured — the filter infers it from how the position changes between steps.

For multiple robots, independent copies of the state vector and filter are run in parallel — one per robot.

**Section 5 — Key Takeaways**

- The LRF produces a polar point cloud $S_k = \{(d_{i,k}, \alpha_{i,k})\}$ at each time step. Noise is modelled as independent Gaussians on range and angle.
- Polar $\rightarrow$ Cartesian uses a nonlinear transform; the **Jacobian** $J$ linearises the noise propagation. The result is a tilted ellipse whose tangential axis grows with range: $\sigma_{\text{tangential}} \approx \bar{d}_i\,\sigma_{\nu_\alpha}$.
- **DBSCAN** clusters the point cloud into groups, one per object. Each cluster is fitted with a **disc** (circle) model via linear least-squares: $p = (XX^\top)^{-1}Xq$, giving centre and radius in one matrix step.
- The disc centre positions are fed into a **Kalman filter** with a constant-velocity state model $\chi_k = [x, y, v_x, v_y]^\top$ to track moving robots and estimate velocity from position measurements alone.
- For multiple robots, independent Kalman filter instances run in parallel — one state vector per robot.